

Глубинное обучение

Лекция 8

Детектирование объектов

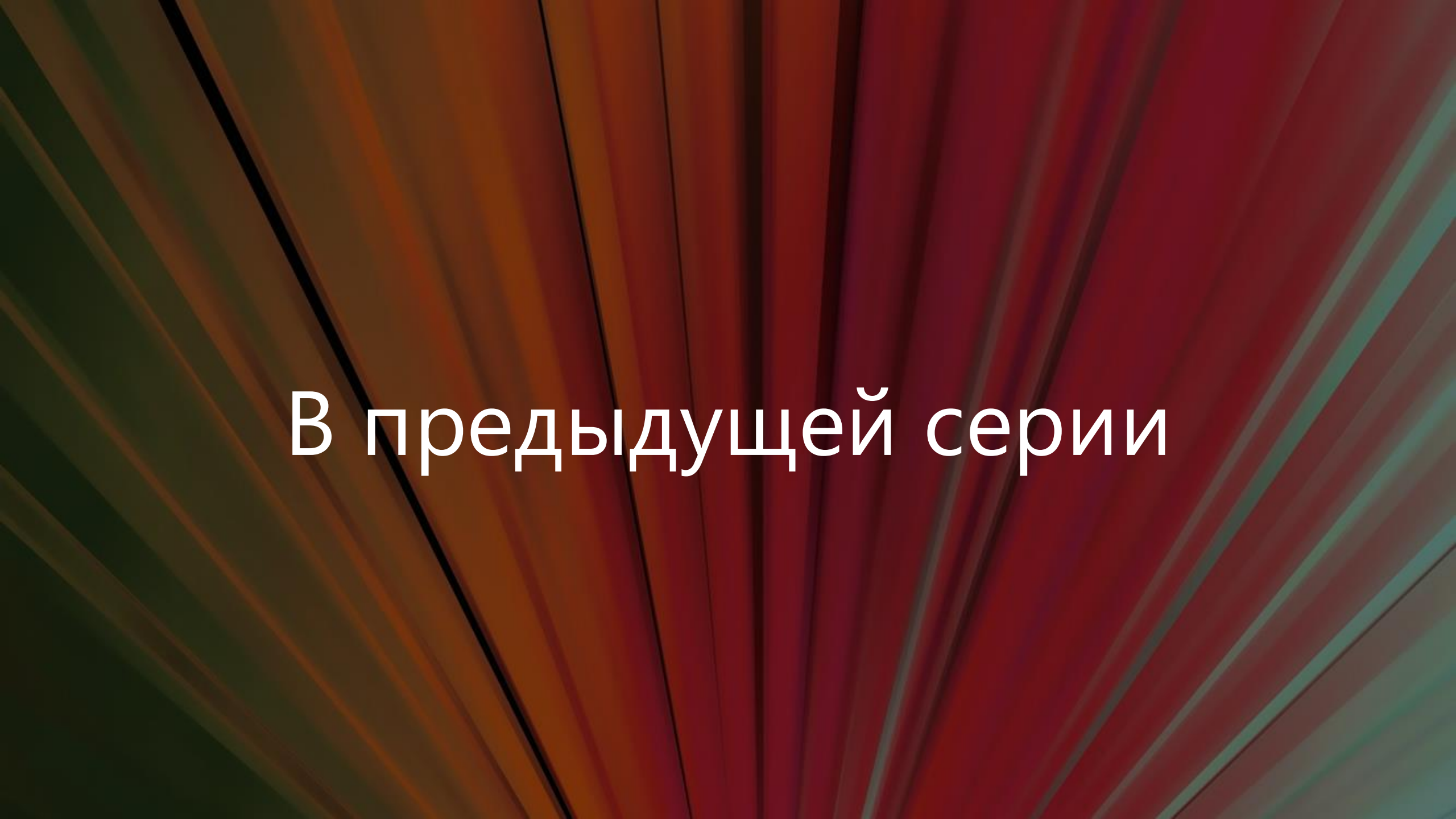
Михаил Гущин

mhushchyn@hse.ru

НИУ ВШЭ, 2026



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ



В предыдущей серии

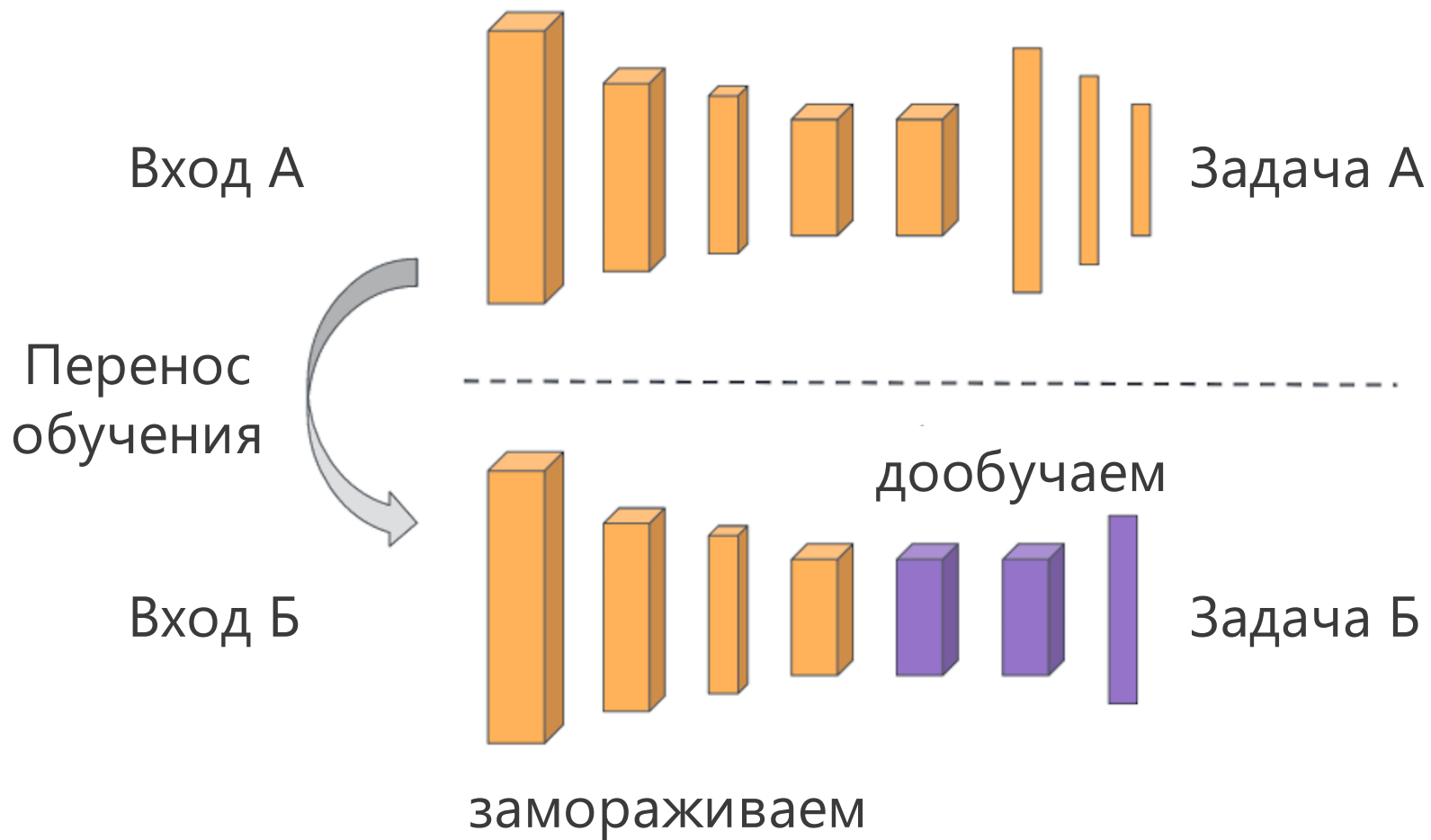
ImageNet



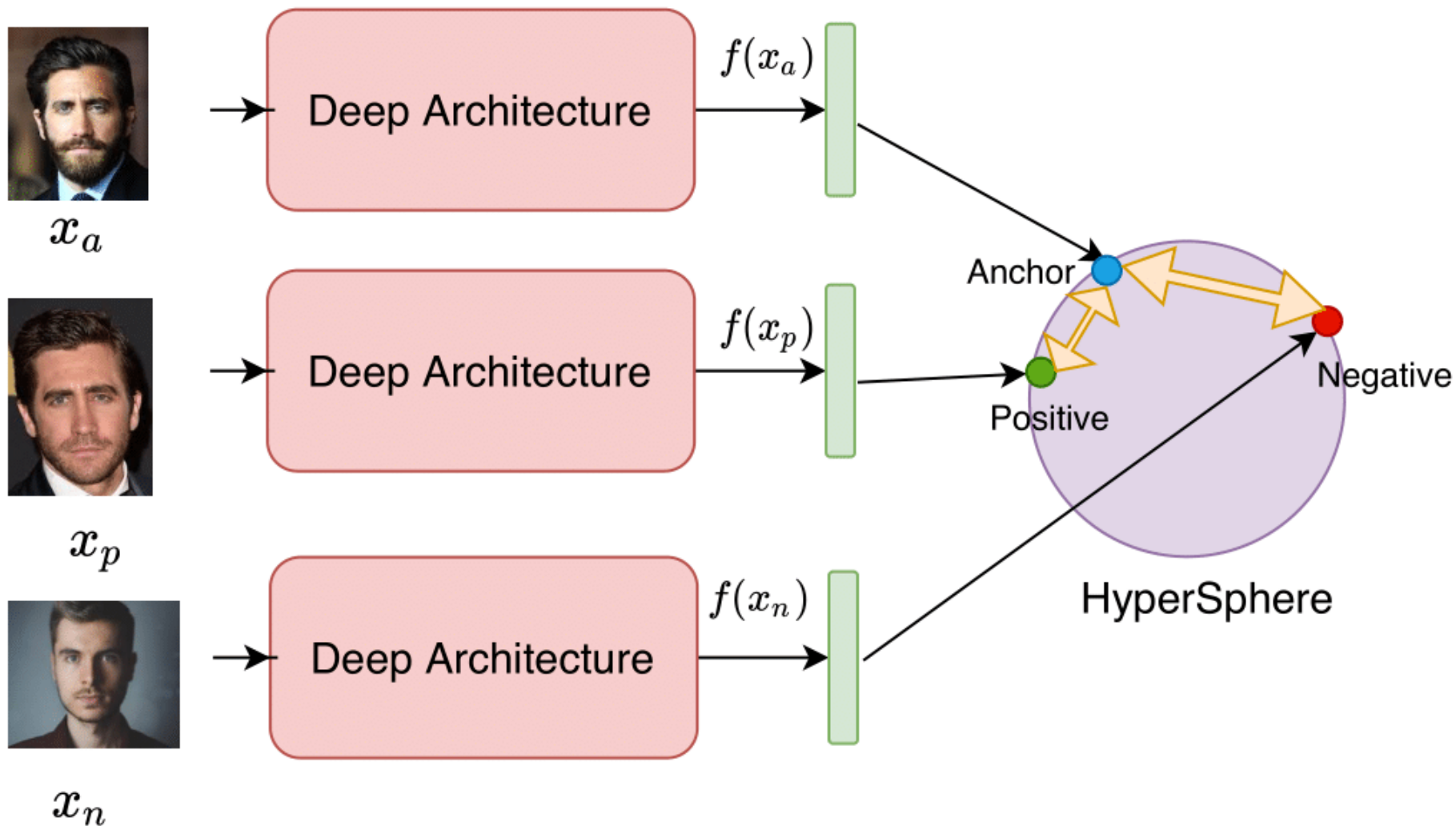
Подробнее: <https://image-net.org/challenges/LSVRC>

- ▶ ImageNet Large Scale Visual Recognition Challenge (ILSVRC)
- ▶ 1000 классов изображений
- ▶ Более 1 000 000 изображений

Перенос обучения (fine-tuning)



Триплет лосс



Источник: <https://pyimagesearch.com/2023/03/06/triplet-loss-with-keras-and-tensorflow/>

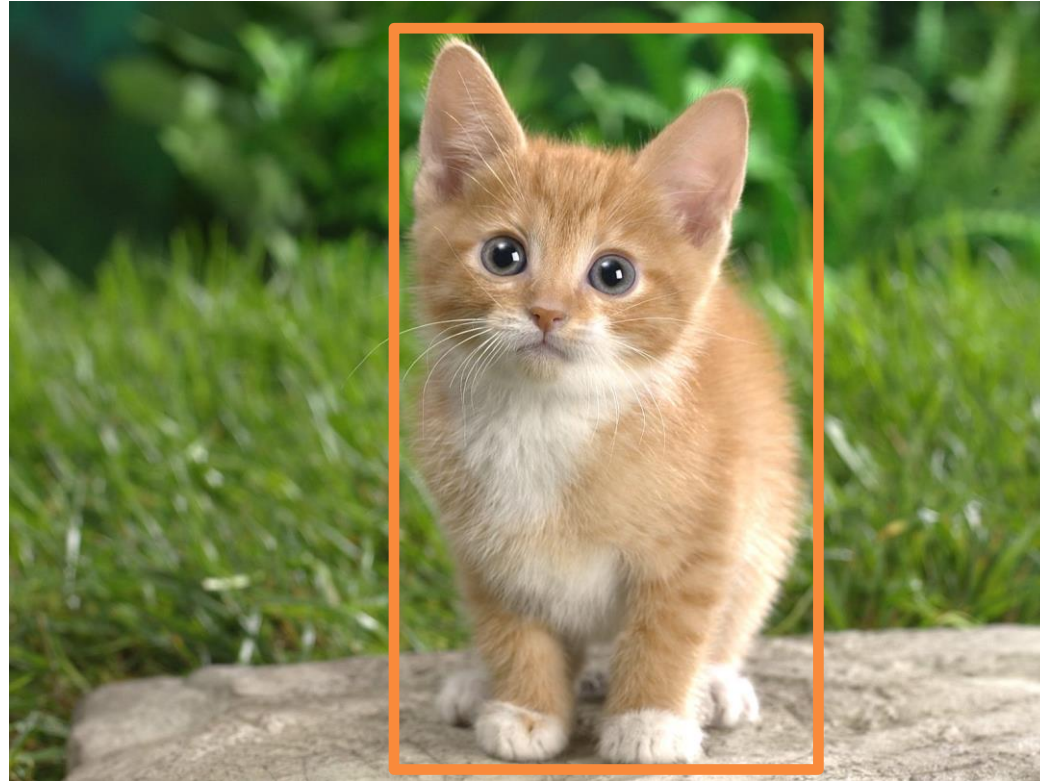
Задачи компьютерного зрения

Классификация изображений



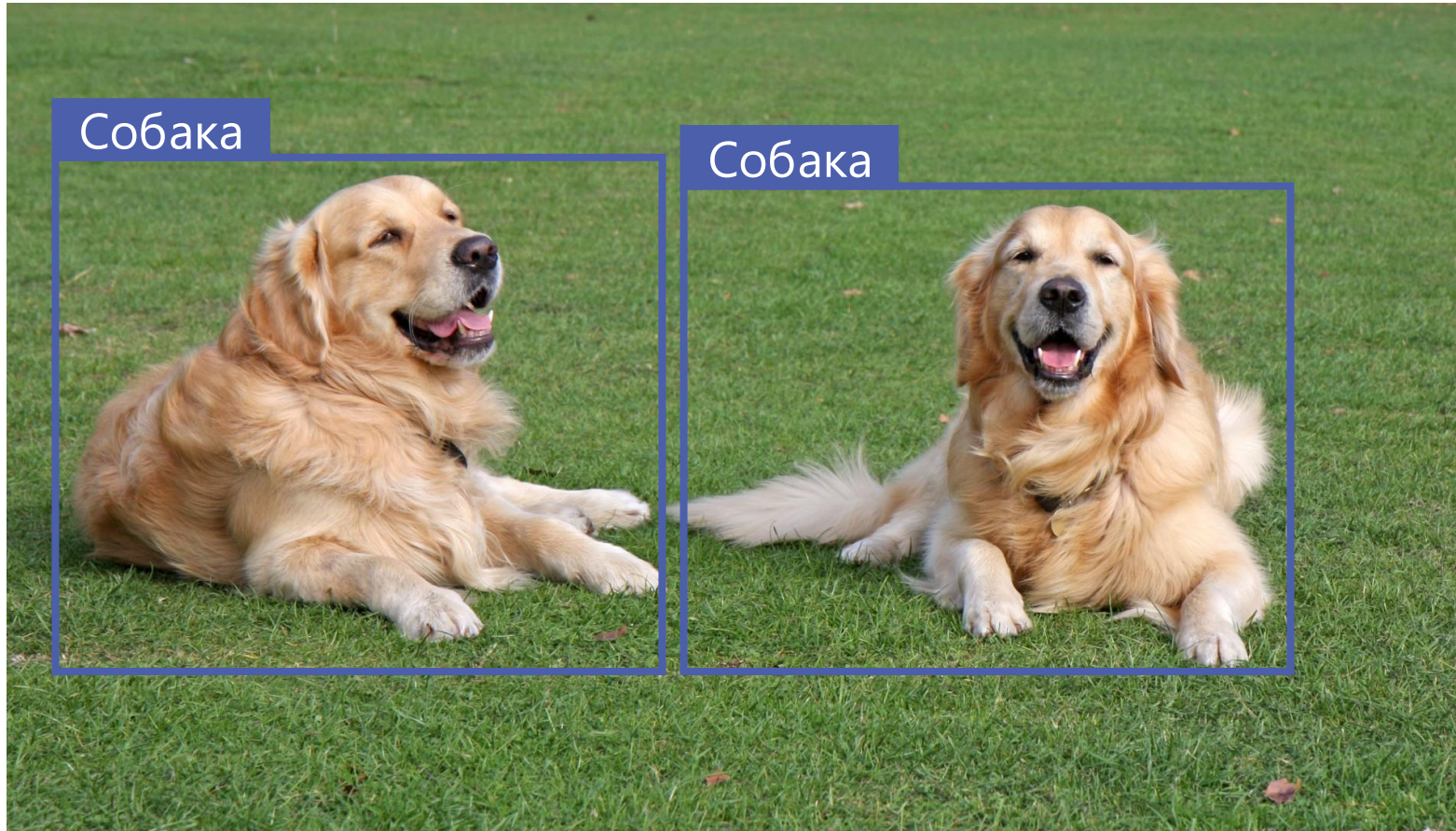
«КОТ»

Классификация и локализация

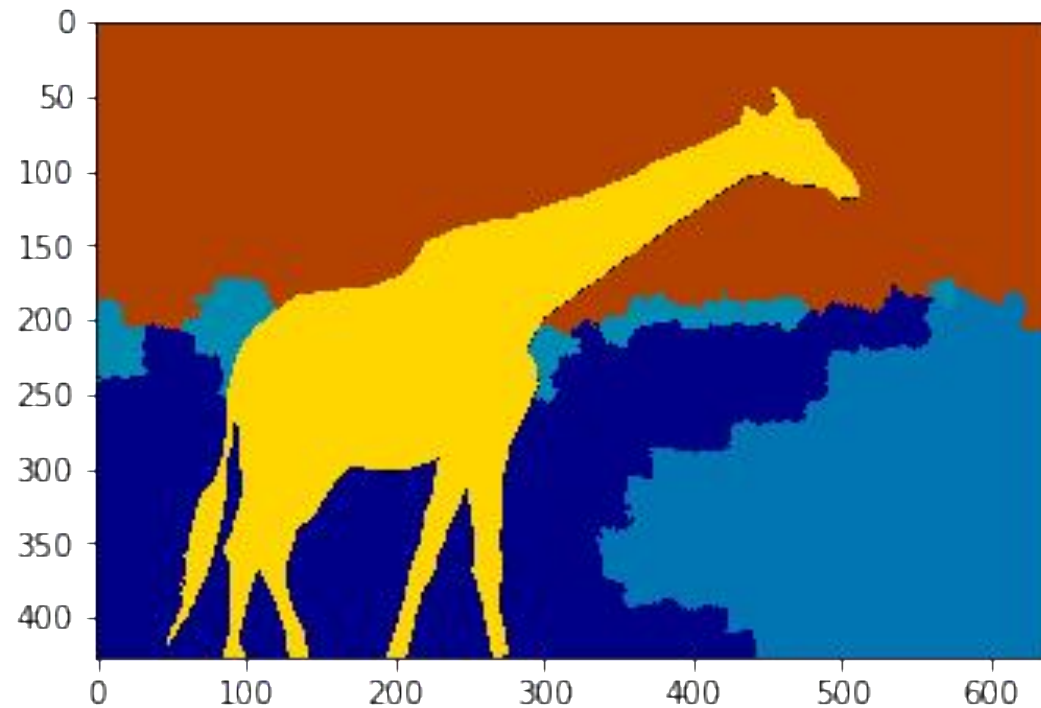
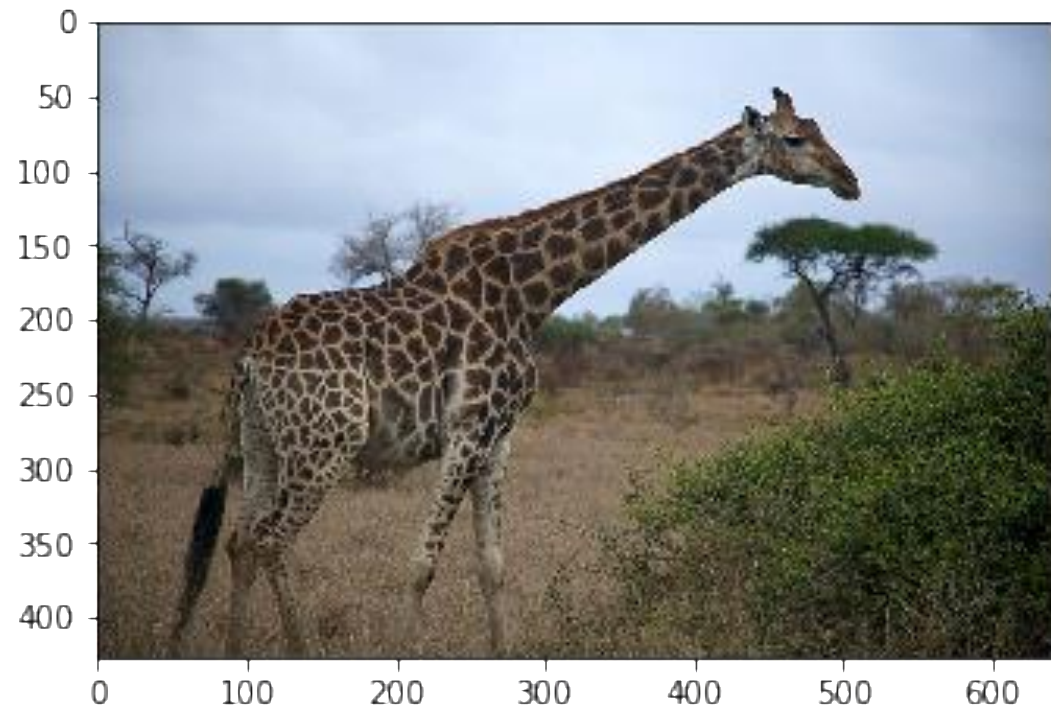


«КОТ»

Детектирование объектов



Семантическая сегментация



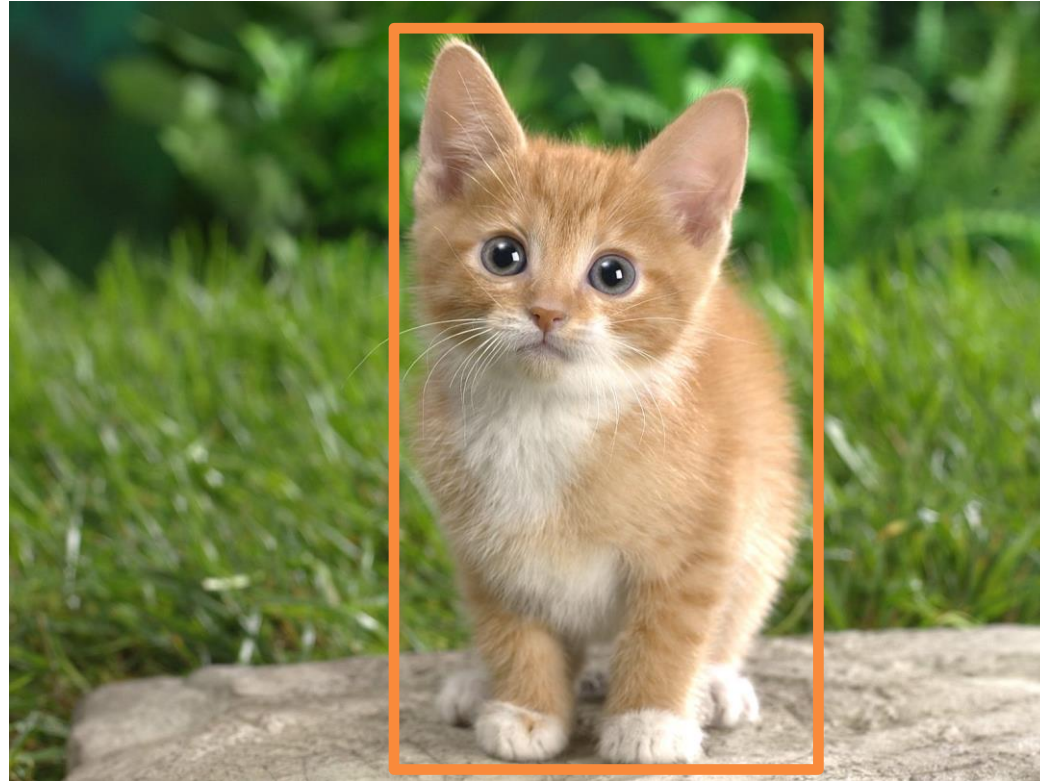
mxnet.apache.org

Локализация объекта

Классификация

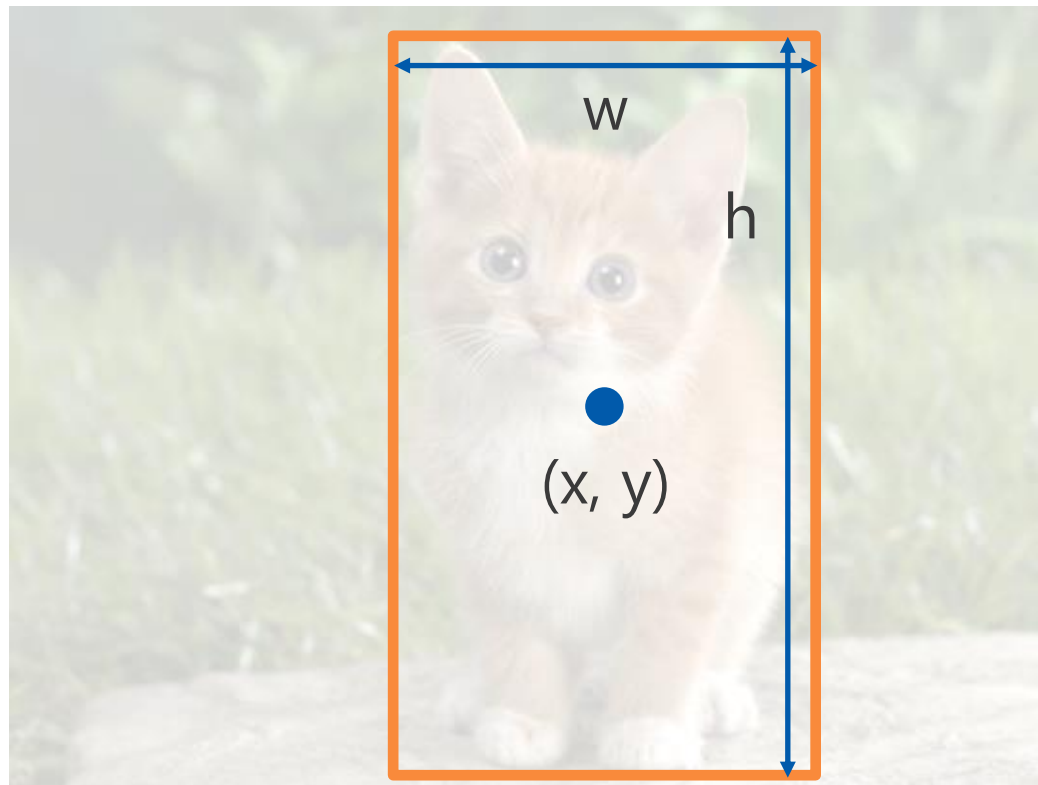


Классификация и локализация



«КОТ»

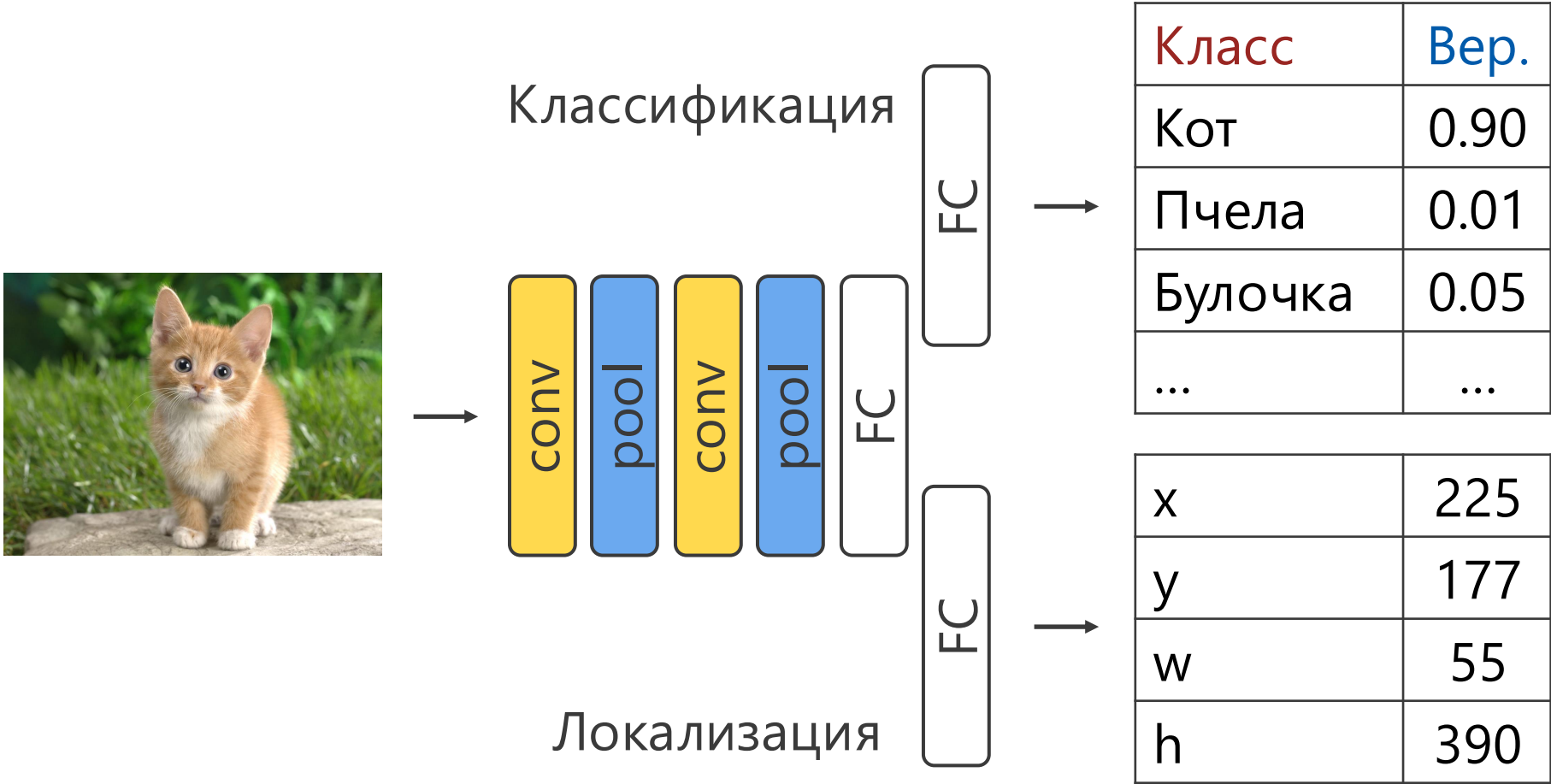
Классификация и локализация



Класс: «Кот»

Прямоугольник: (x, y, w, h)

Классификация и локализация



Перенос обучения

- ▶ За основу берем сеть, обученную на ImageNet (VGG-16)
- ▶ Добавляем голову для локализации
- ▶ Дообучаем последние слои сети

Функция потерь

Многозадачная функция потерь L :

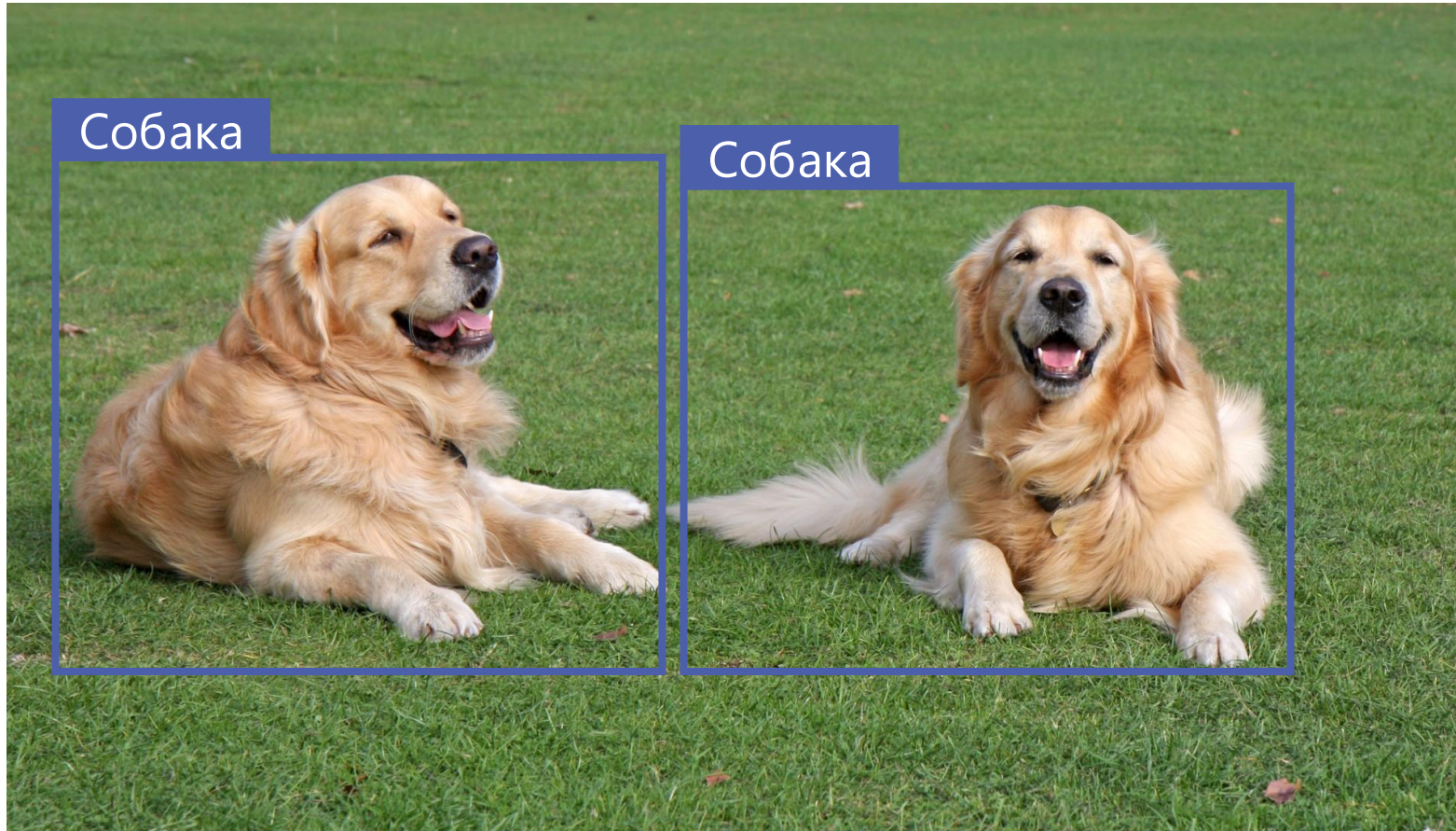
$$L = L_{\text{классификация}} + \alpha L_{\text{локализация}} \rightarrow \min$$

- ▶ $L_{\text{классификация}}$ — функция кросс-энтропии для классификации изображения
- ▶ $L_{\text{локализация}}$ — MSE функция потерь для параметров прямоугольника



R-CNN

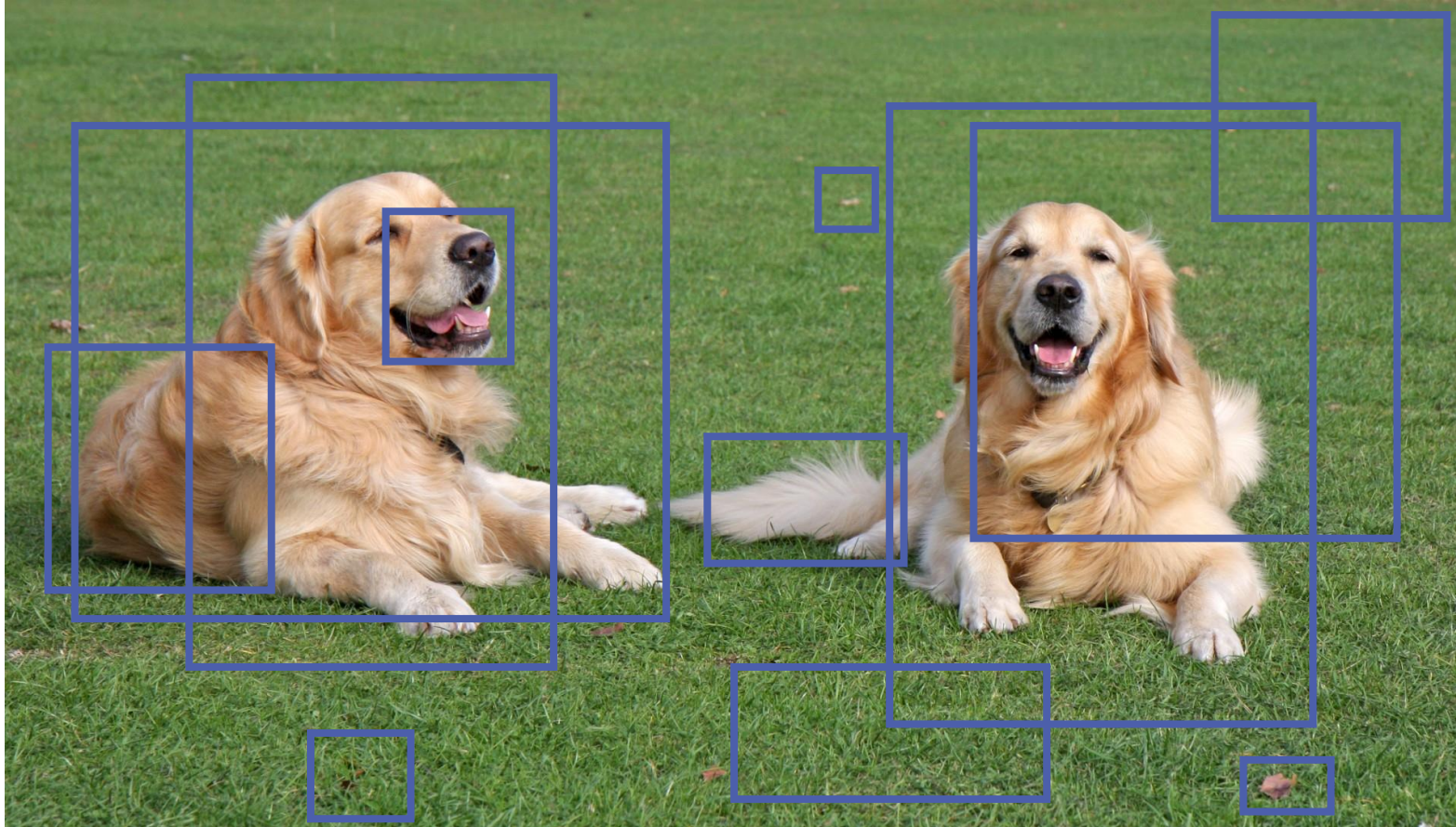
Детектирование объектов



Идея R-CNN алгоритма

- ▶ Генерация областей изображения (region proposals)
- ▶ Классификация с локализацией объектов для каждой области
- ▶ Отбор лучших прямоугольников (x, y, w, h) (non-maximum suppression)

Region proposal



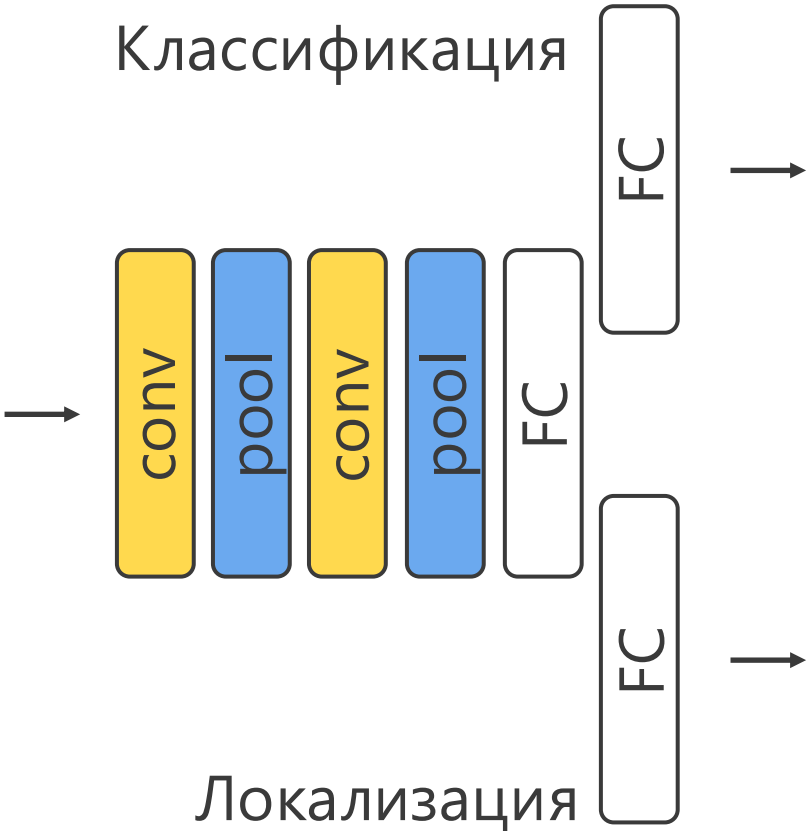
Выбираем около 2000 областей

Region proposal

- ▶ Sliding window
 - последовательный перебор всех областей разных размеров;
 - простой, но неэффективный.
- ▶ Selective search
 - иерархическое объединение областей по цвету, текстуре, размеру и форме;
 - популярный и эффективный алгоритм.

Классификация + локализация для регионов

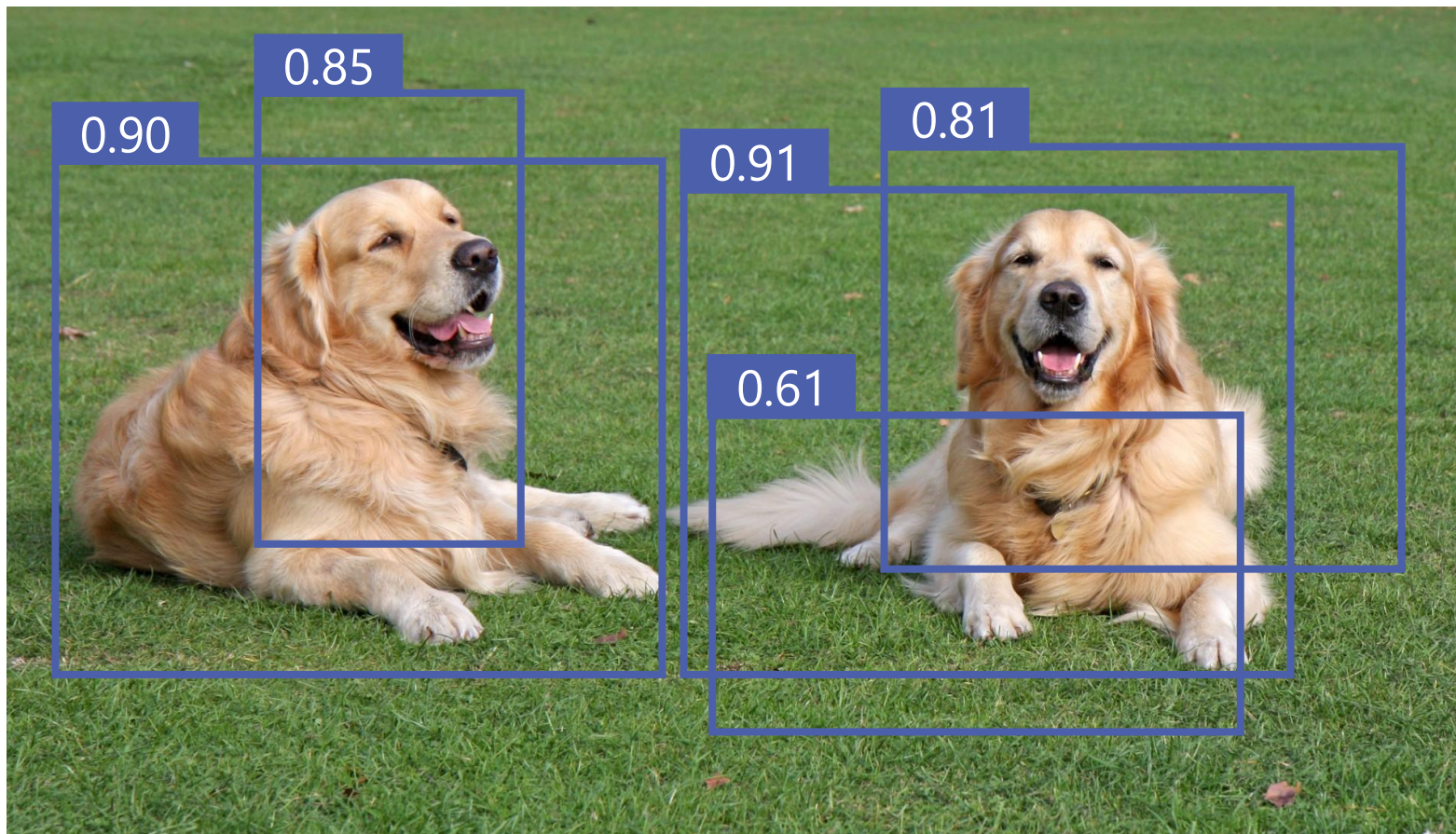
Область
изображения



Класс	Вер.
Собака	0.90
Пчела	0.01
Булочка	0.05
...	...

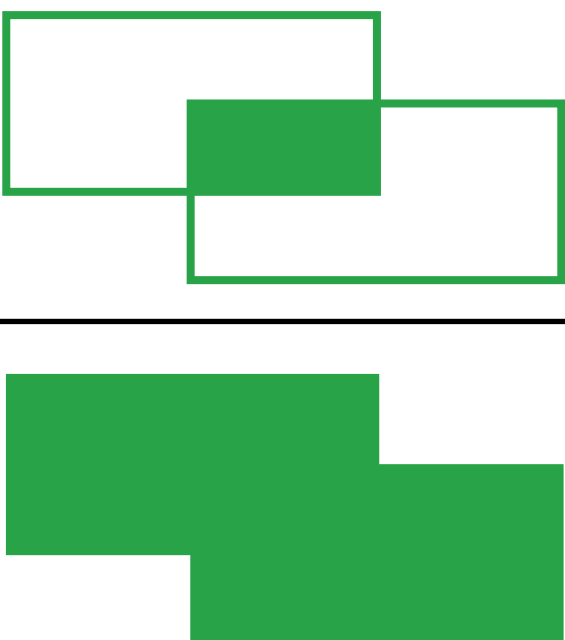
x	225
y	177
w	55
h	390

Классификация + локализация для регионов



Как выбрать нужные прямоугольники?

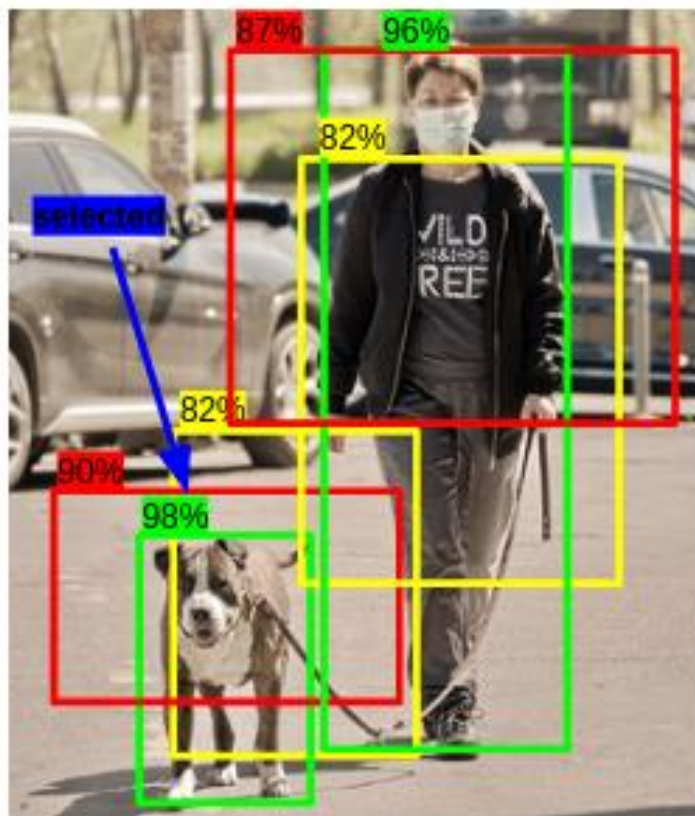
Intersection over Union (IoU)

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$


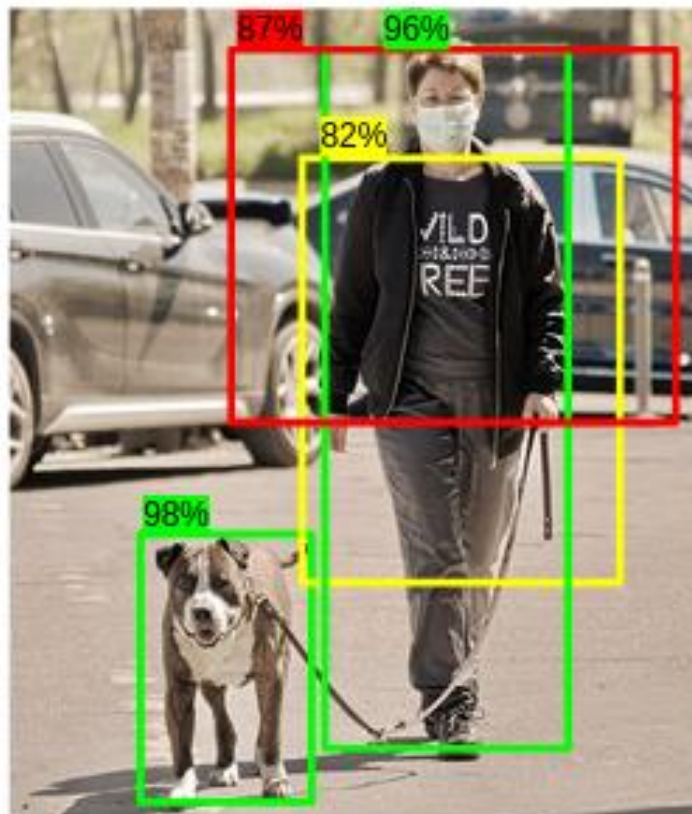
Non-maximum suppression

- ▶ Для **каждого класса** берем прямоугольник (x, y, w, h) с наибольшей вероятностью;
- ▶ Удаляем прямоугольники с меньшими вероятностями, для которых $\text{IoU} > k$;
- ▶ Повторяем шаги, пока не переберем все прямоугольники.

Пример



Step 1: Selecting Bounding box with highest score

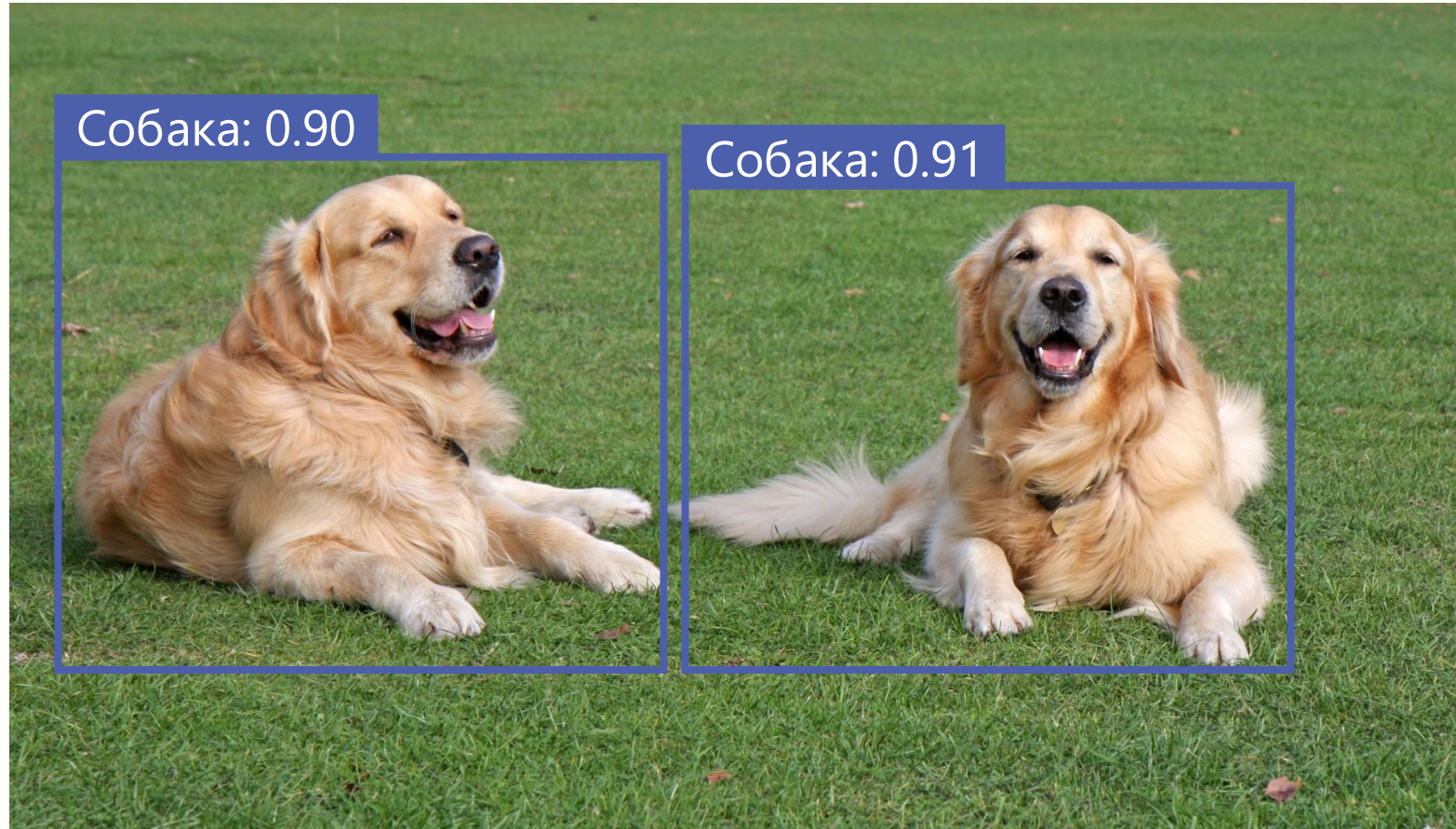


Step 3: Delete Bounding box with high overlap



Step 5: Final Output

Пример



R-CNN алгоритм

- ▶ Генерация областей (region proposals)
- ▶ Классификация с локализацией объекта для каждого прямоугольника
- ▶ Отбор лучших прямоугольников (x, y, w, h) (non-maximum suppression)

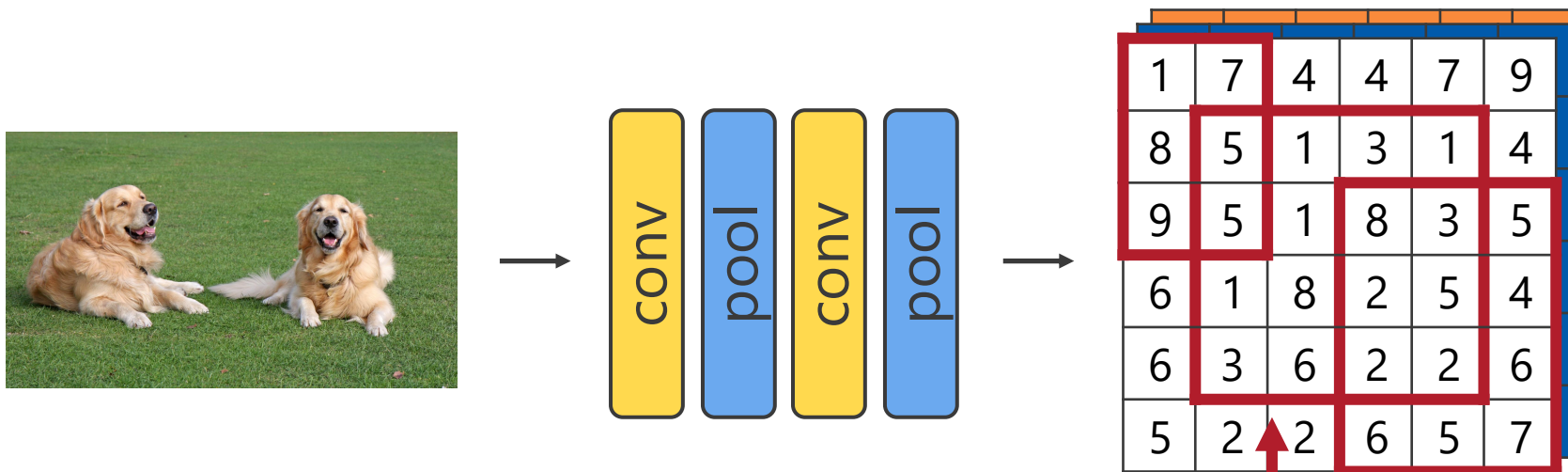


Fast R-CNN

R-CNN

- ▶ Простой и эффективный
- ▶ Около 2000 областей для классификации
- ▶ Долго обрабатывает одно изображение
- ▶ Как можно ускорить?

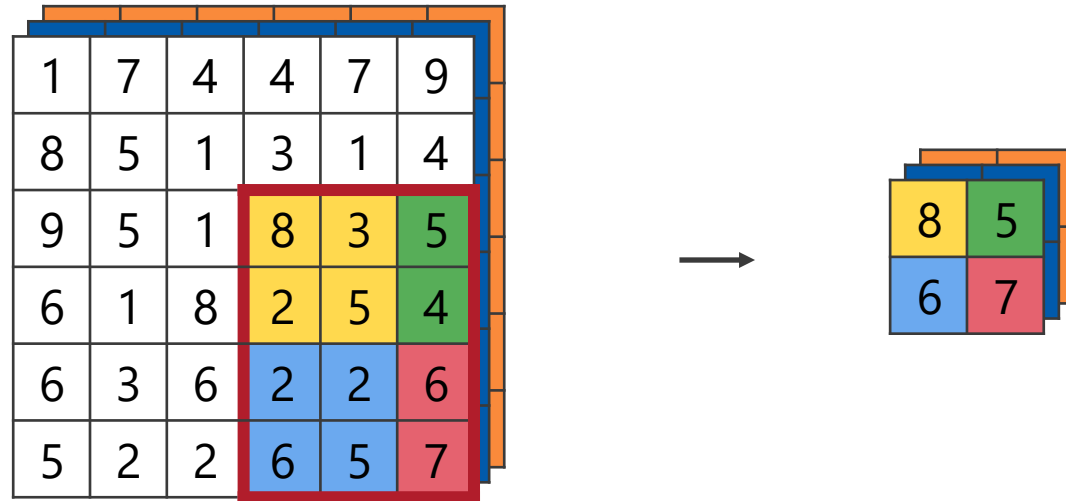
Region proposals



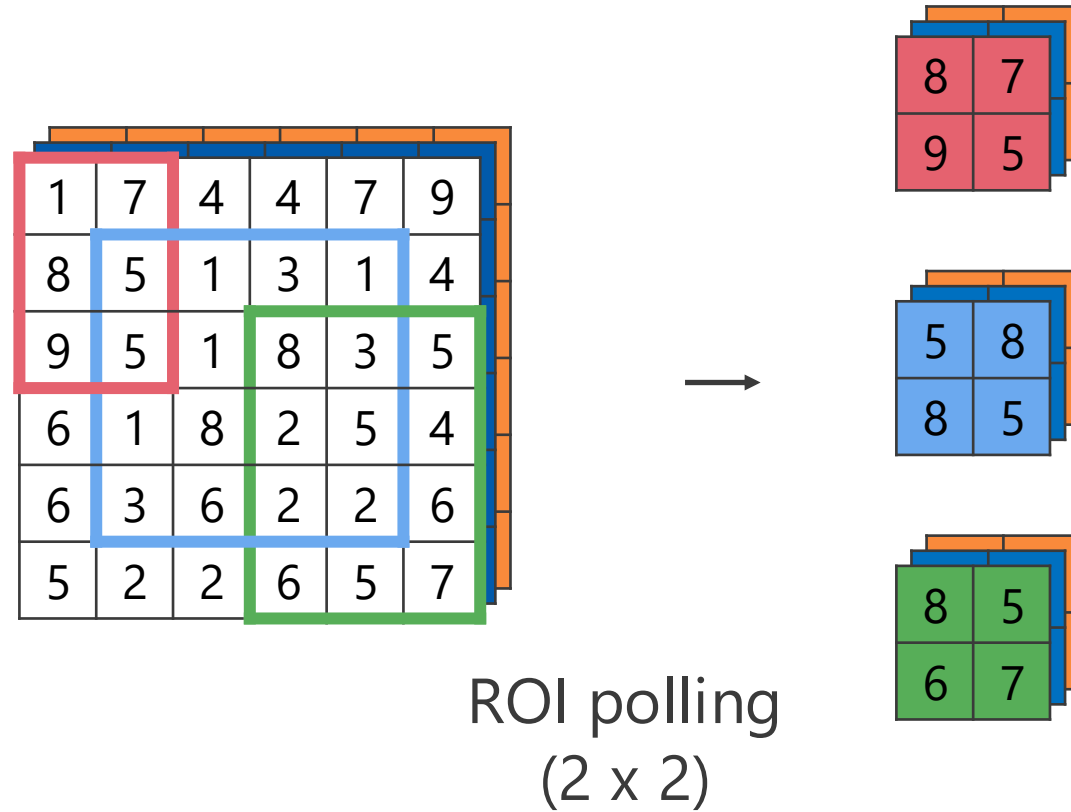
Сгенерированные области, ROIs
(Regions of Interest)

ROI pooling

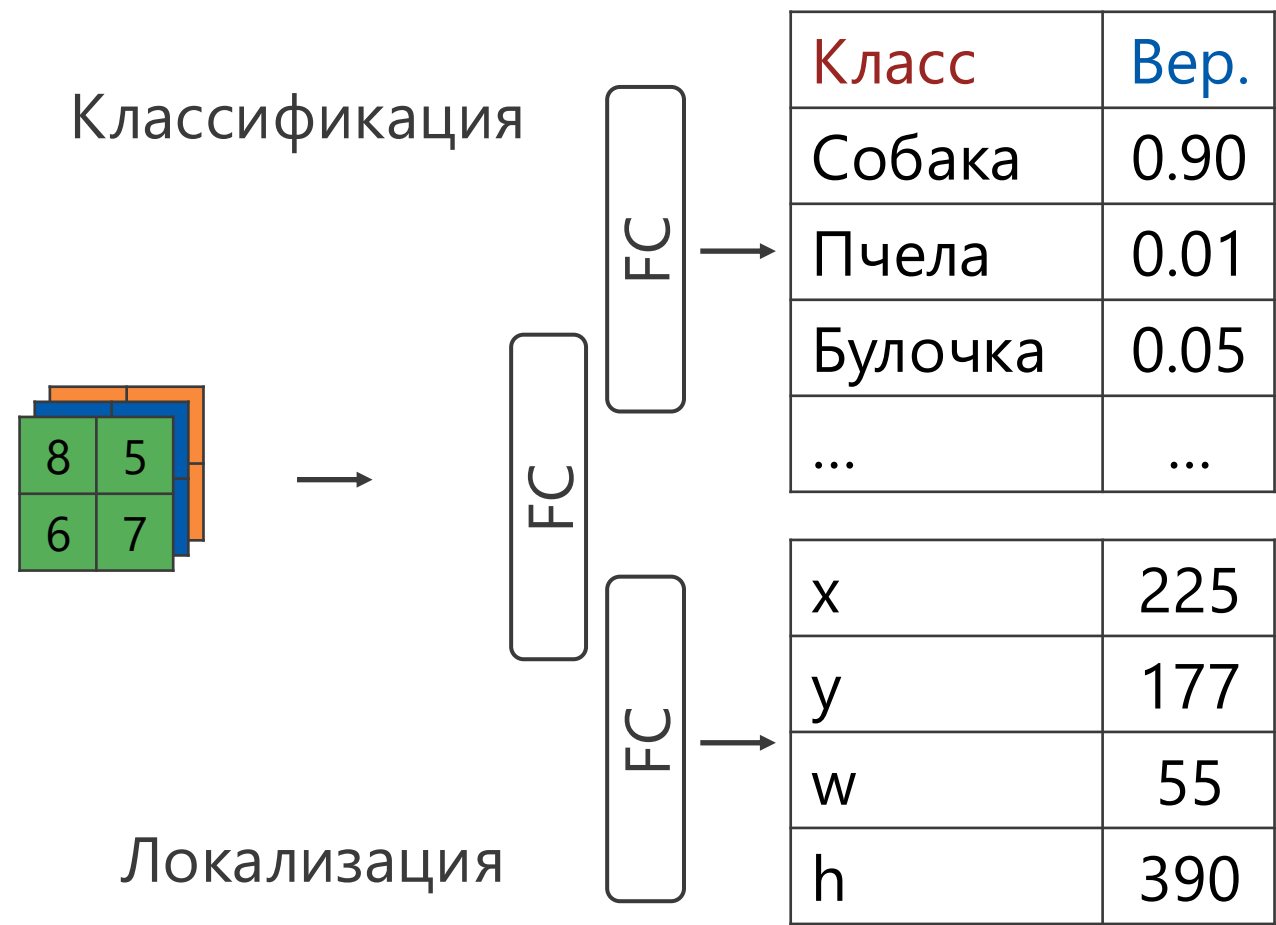
- ▶ Каждый ROI делим сеткой (2 x 2)
- ▶ В каждой ячейке сетки находим максимум



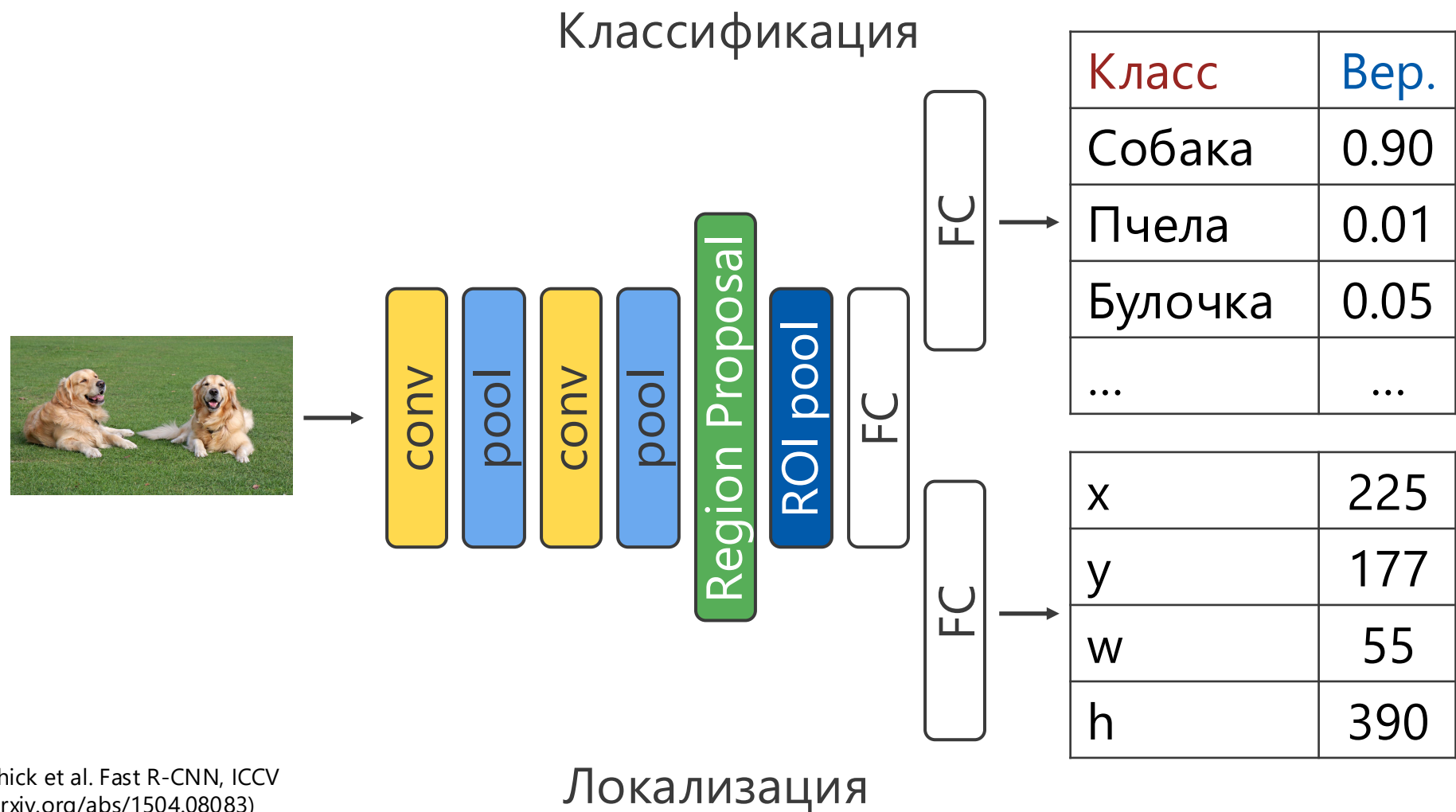
ROI pooling



Классификация + локализация

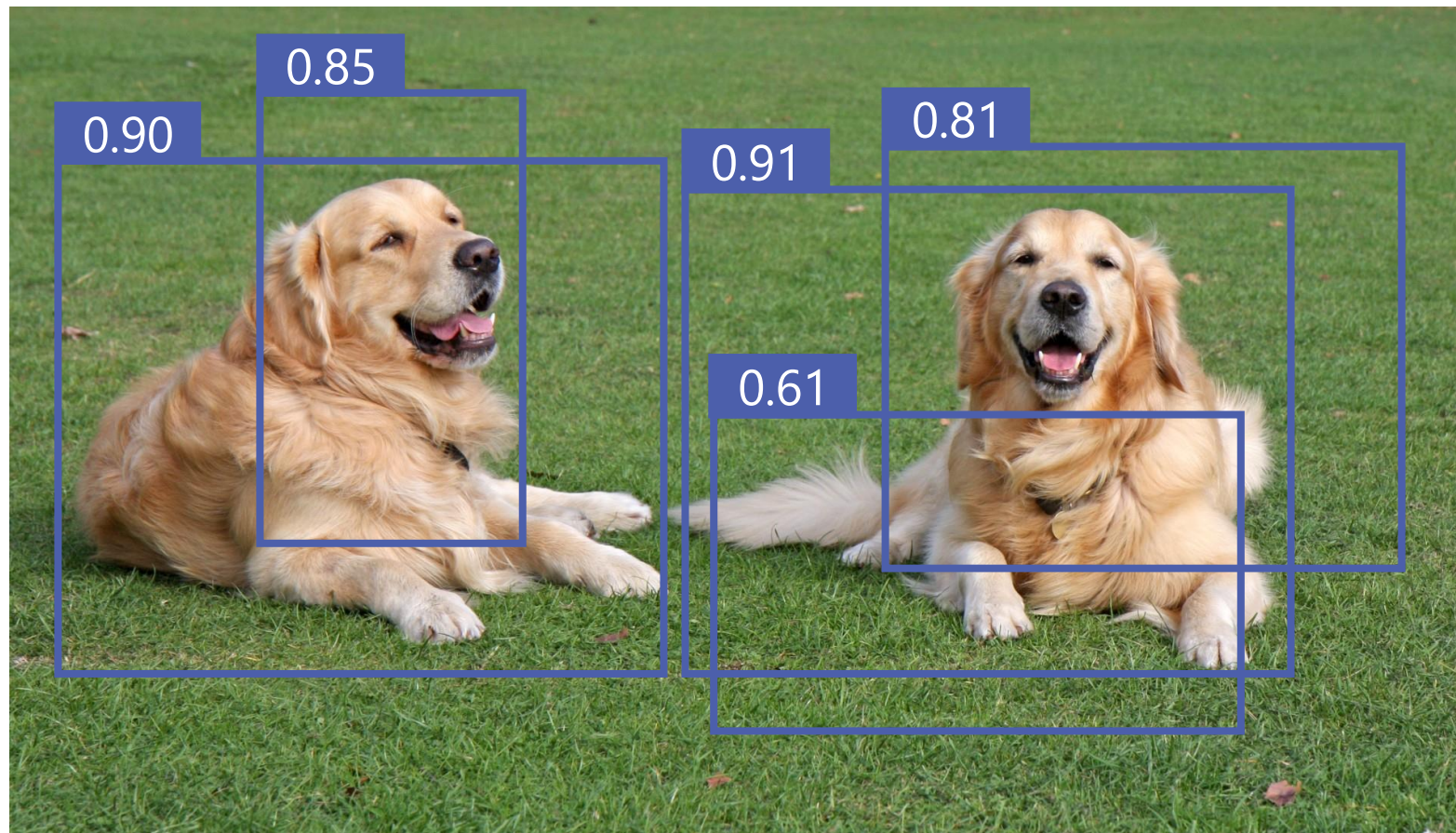


Fast R-CNN



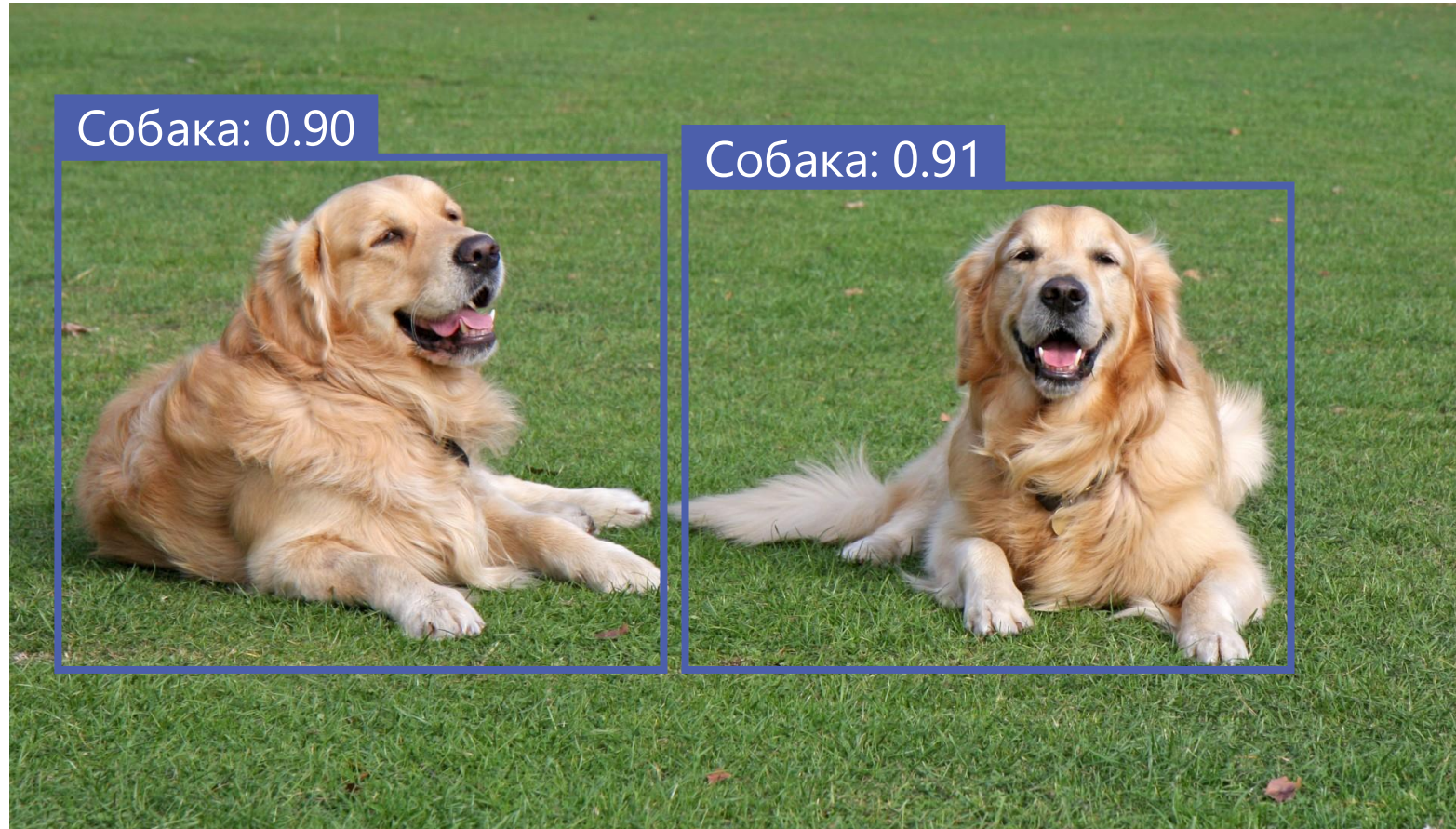
Подробнее: R. Girshick et al. Fast R-CNN, ICCV 2015 (URL: <https://arxiv.org/abs/1504.08083>)

Классификация + локализация для регионов



Как выбрать нужные прямоугольники?

Пример

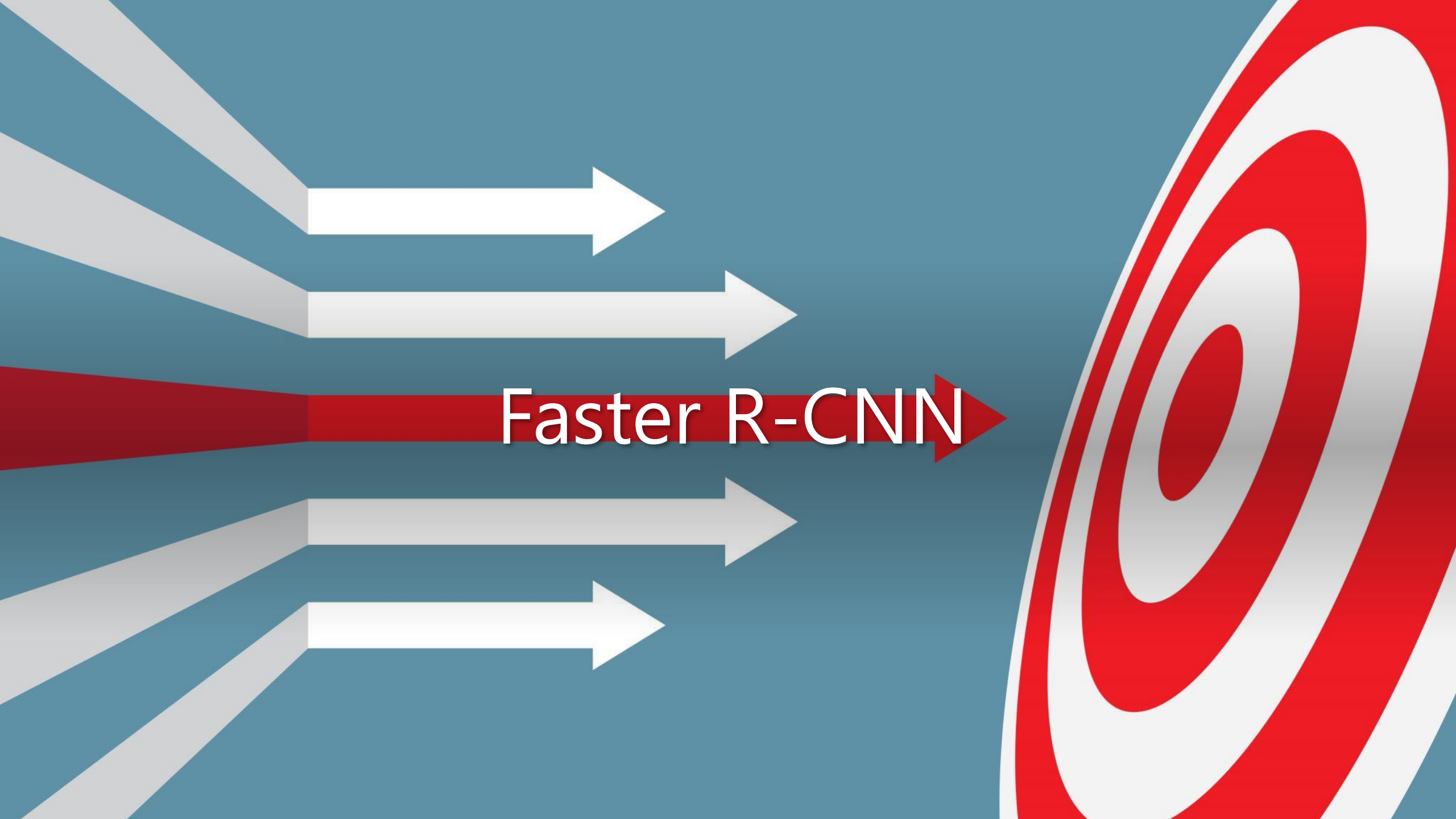


Fast R-CNN

- ▶ Пропускаем изображение через сверточные слои
- ▶ Генерация области (ROI) на свернутом изображении
- ▶ ROI pooling
- ▶ Классификация с локализацией объекта для каждого ROI
- ▶ Отбор лучших прямоугольников (x, y, w, h) (non-maximum suppression)

Выводы

- ▶ Изображение целиком подается на вход модели
- ▶ Изображение проходит через сверточные слои сети только один раз
- ▶ Ускорение в 25 раз по сравнению с R-CNN

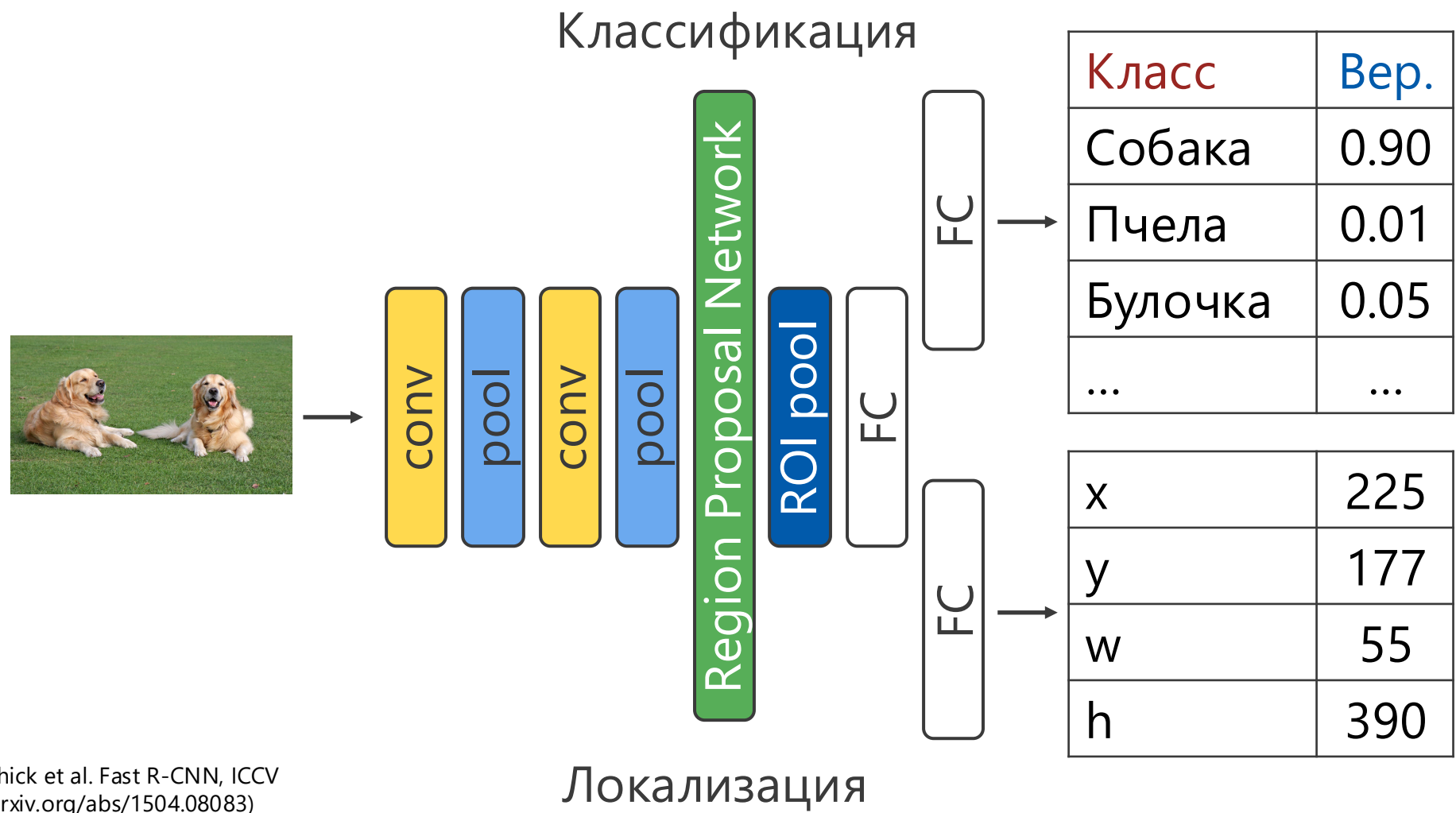
A diagram illustrating the accuracy of the Faster R-CNN model. On the left, several gray arrows of varying widths point towards a target on the right. The central arrow is red and significantly wider than the others, representing higher accuracy. The target is a bullseye with red and white concentric rings. The background is a solid blue color.

Faster R-CNN

Fast R-CNN

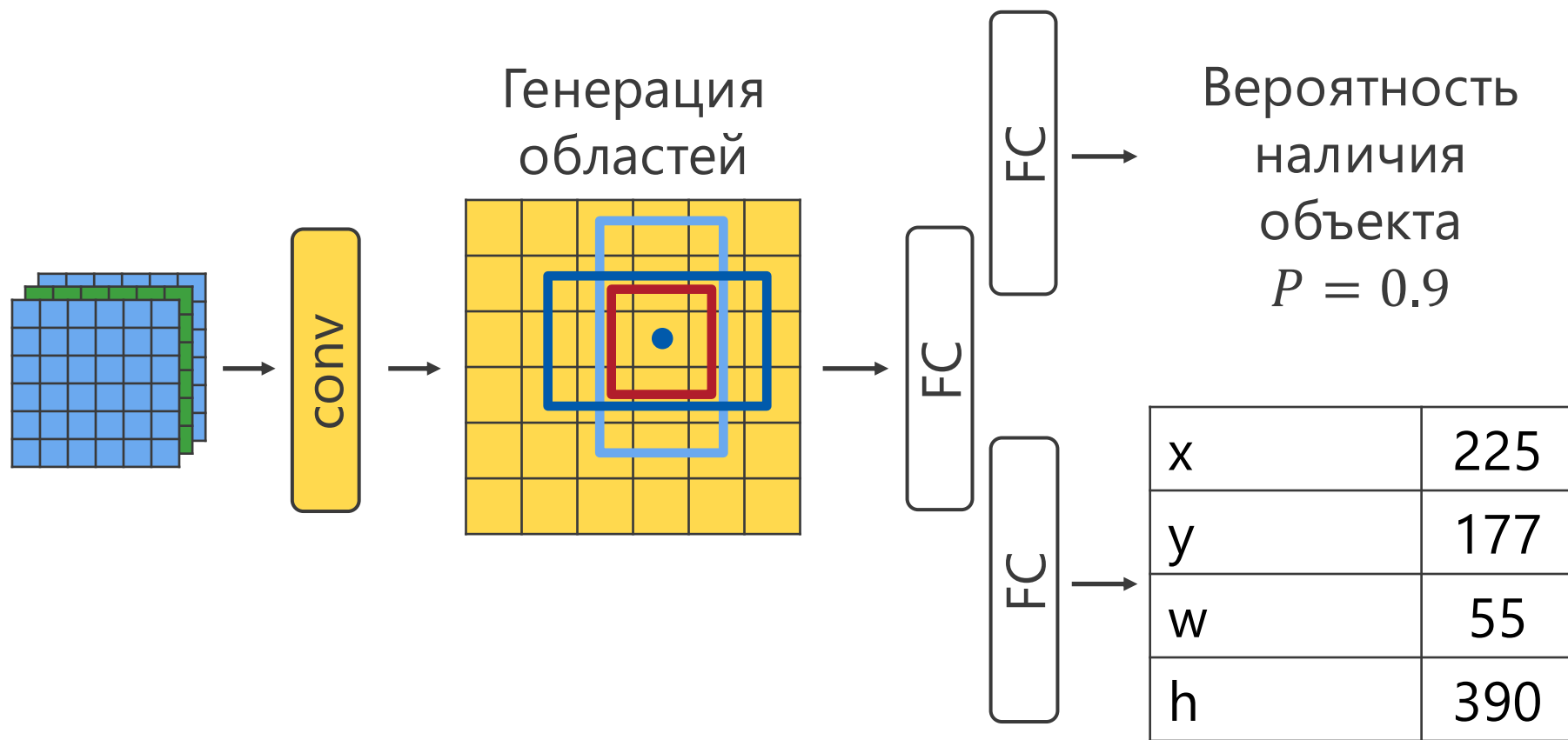
- ▶ Быстрее R-CNN в 25 раз
- ▶ Region proposal занимает больше всего времени
- ▶ Как можно ускорить?

Faster R-CNN



Подробнее: R. Girshick et al. Fast R-CNN, ICCV 2015 (URL: <https://arxiv.org/abs/1504.08083>)

Region proposal network



* Обучаем эту сеть отдельно

Region proposal network

- ▶ Простая генерация областей
- ▶ Оставляем только области, где предсказан объект
- ▶ Применяем non-maximum suppression для отбора лучших областей
- ▶ Выводим предсказанные локализации (x, y, w, h) в лучших областях
- ▶ Локализации (x, y, w, h) используем как ROI в Faster R-CNN

	R-CNN	Fast R-CNN	Faster R-CNN
Время на изображение	50 сек	2 сек	0.2 сек
Ускорение	1	25	250
mAP (VOC 2007)	66	66.9	66.9

* mAP — mean Average Precision

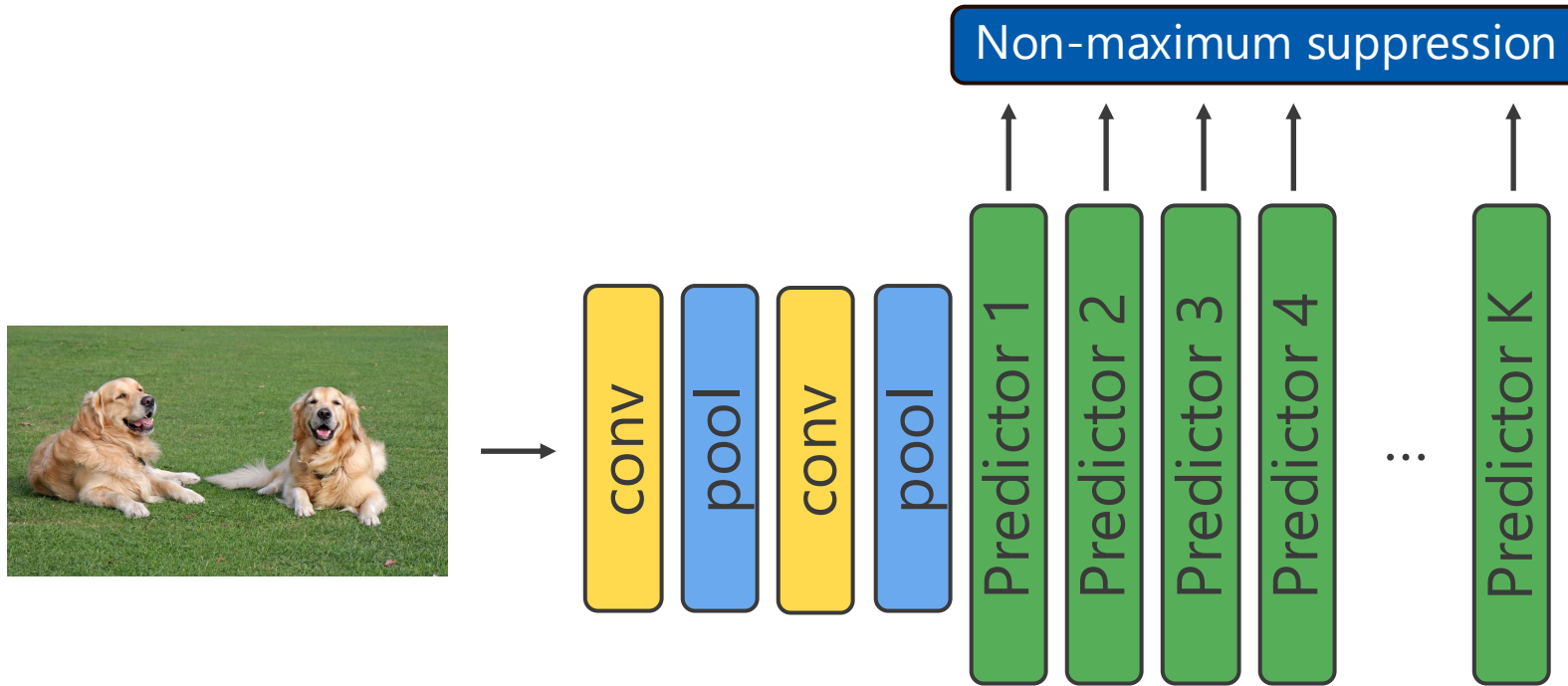


SSD: Single Shot MultiBox Detector

R-CNN

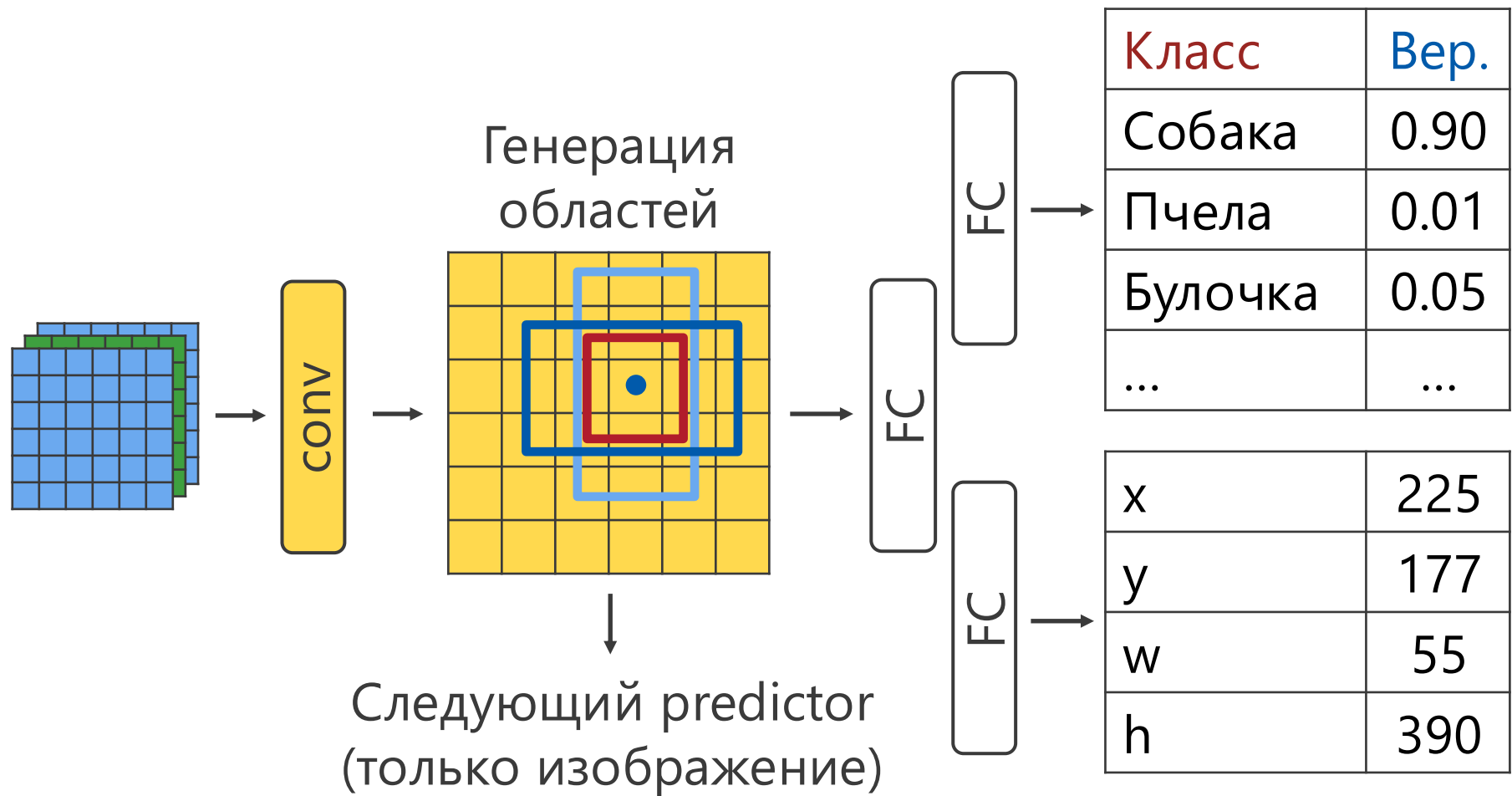
- ▶ Два этапа:
 - Сложная генерация областей
 - Классификация и локализация внутри каждой области
- ▶ Высокое качество
- ▶ Медленные
- ▶ Можно ли обойтись одним этапом?

SSD: Single Shot MultiBox Detector



Подробнее: W. Liu et al. SSD: Single Shot MultiBox Detector, ECCV 2016 (URL: <https://arxiv.org/abs/1512.02325>)

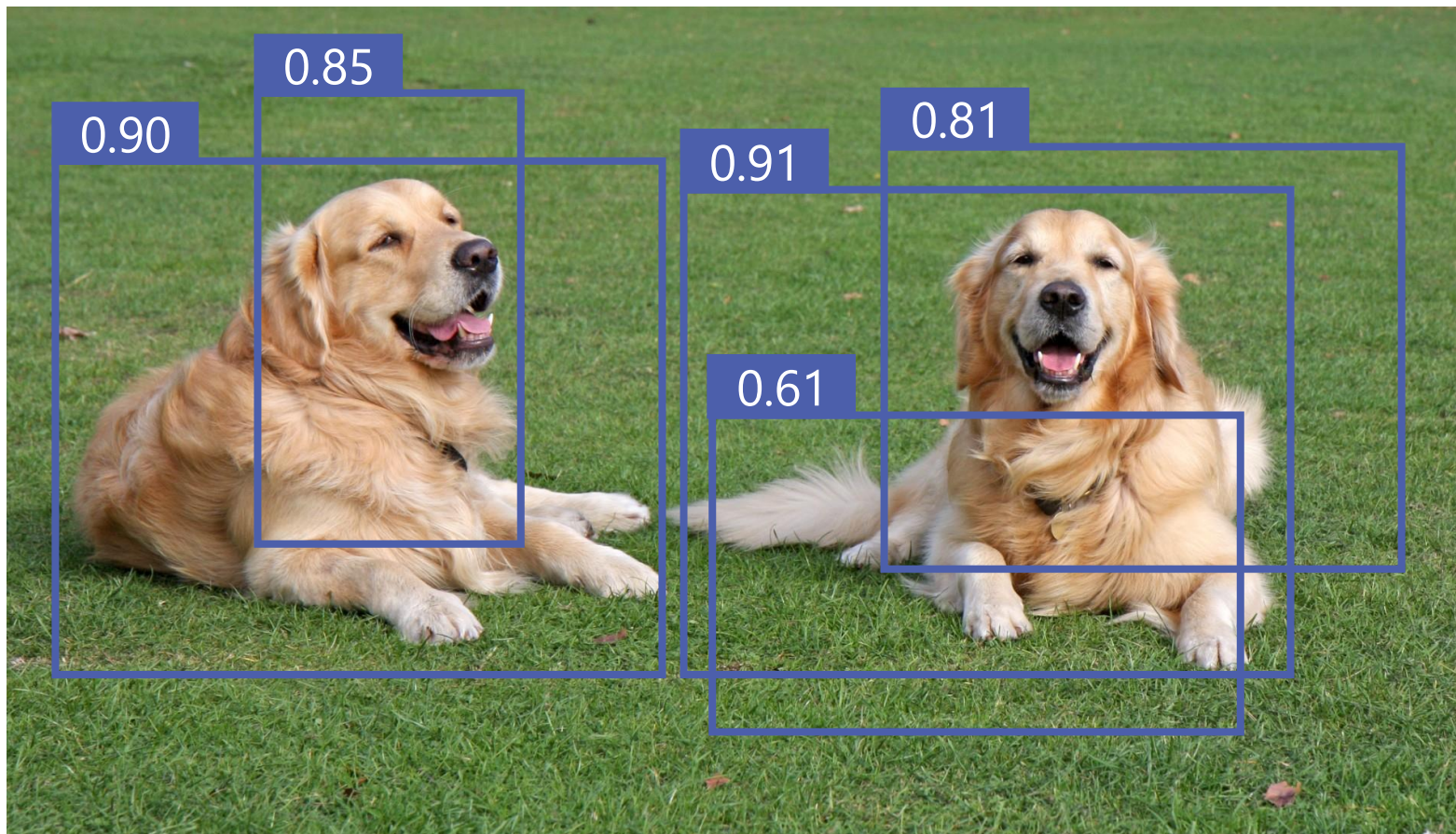
Predictor



SSD: Single Shot MultiBox Detector

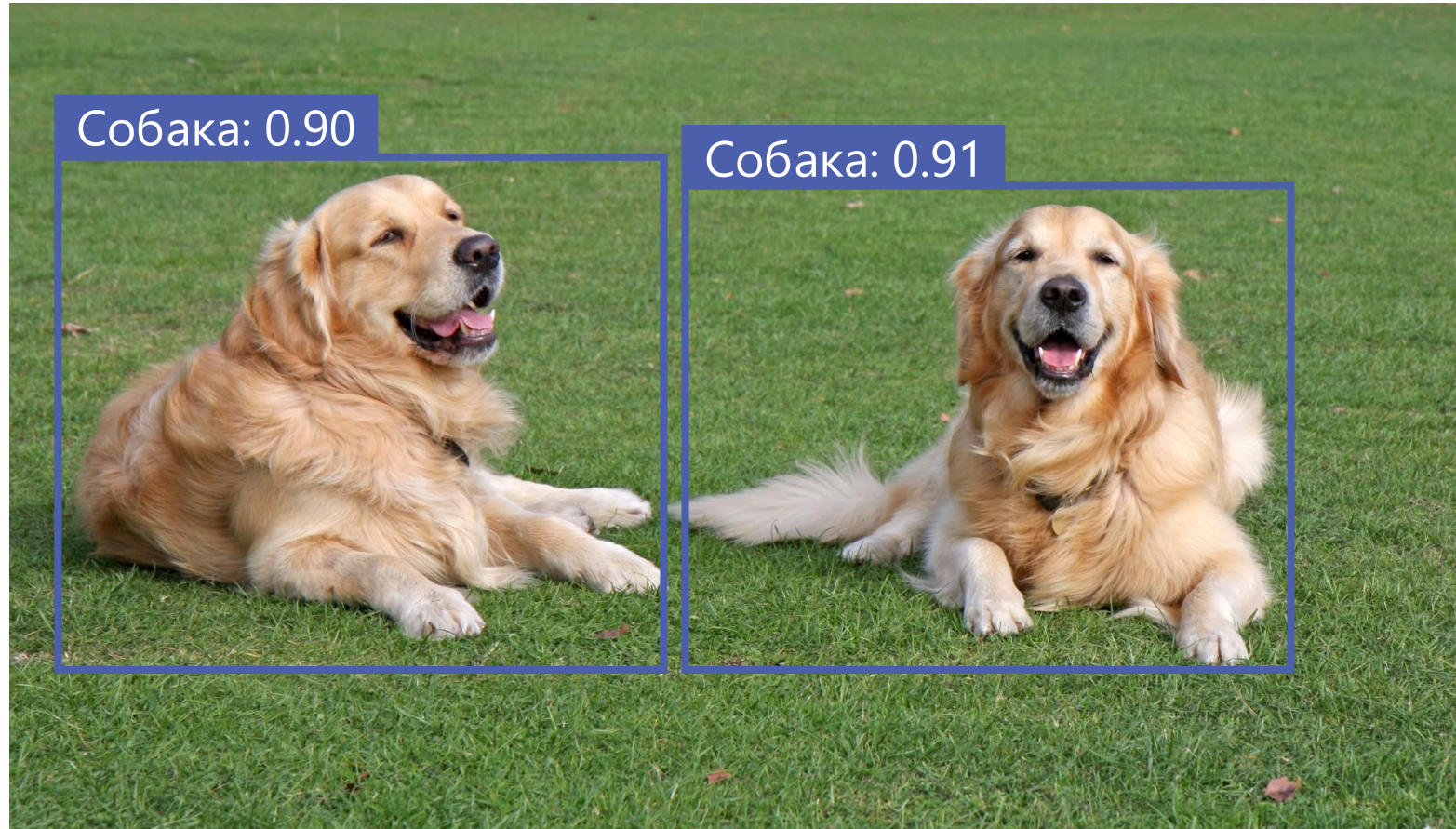
- ▶ Каждый предиктор сворачивает изображение
- ▶ Классификация и локализация в предикторе применяются к изображениям разных размеров
- ▶ Предсказания всех предикторов объединяются

Классификация + локализация для регионов

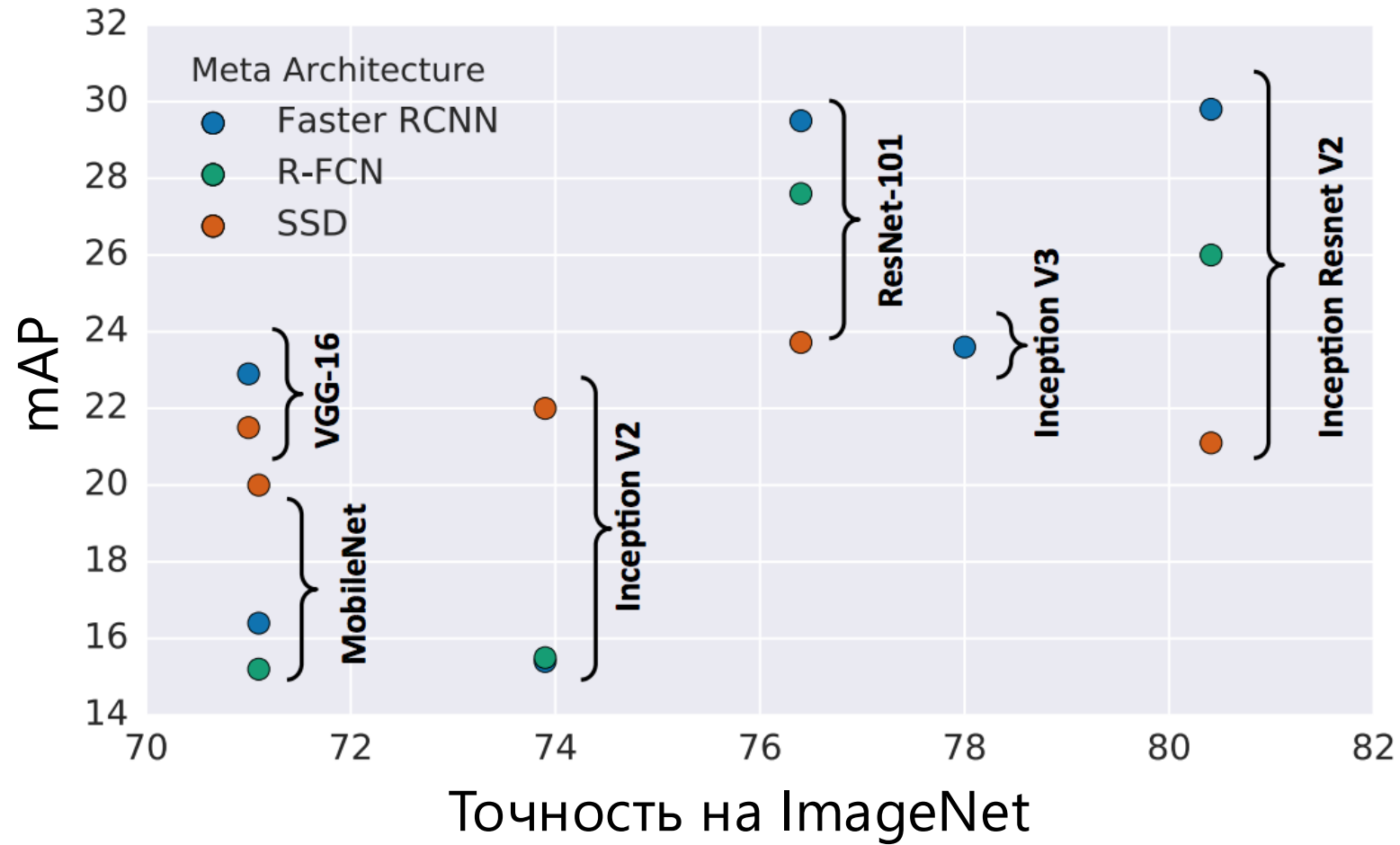


Как выбрать нужные прямоугольники?

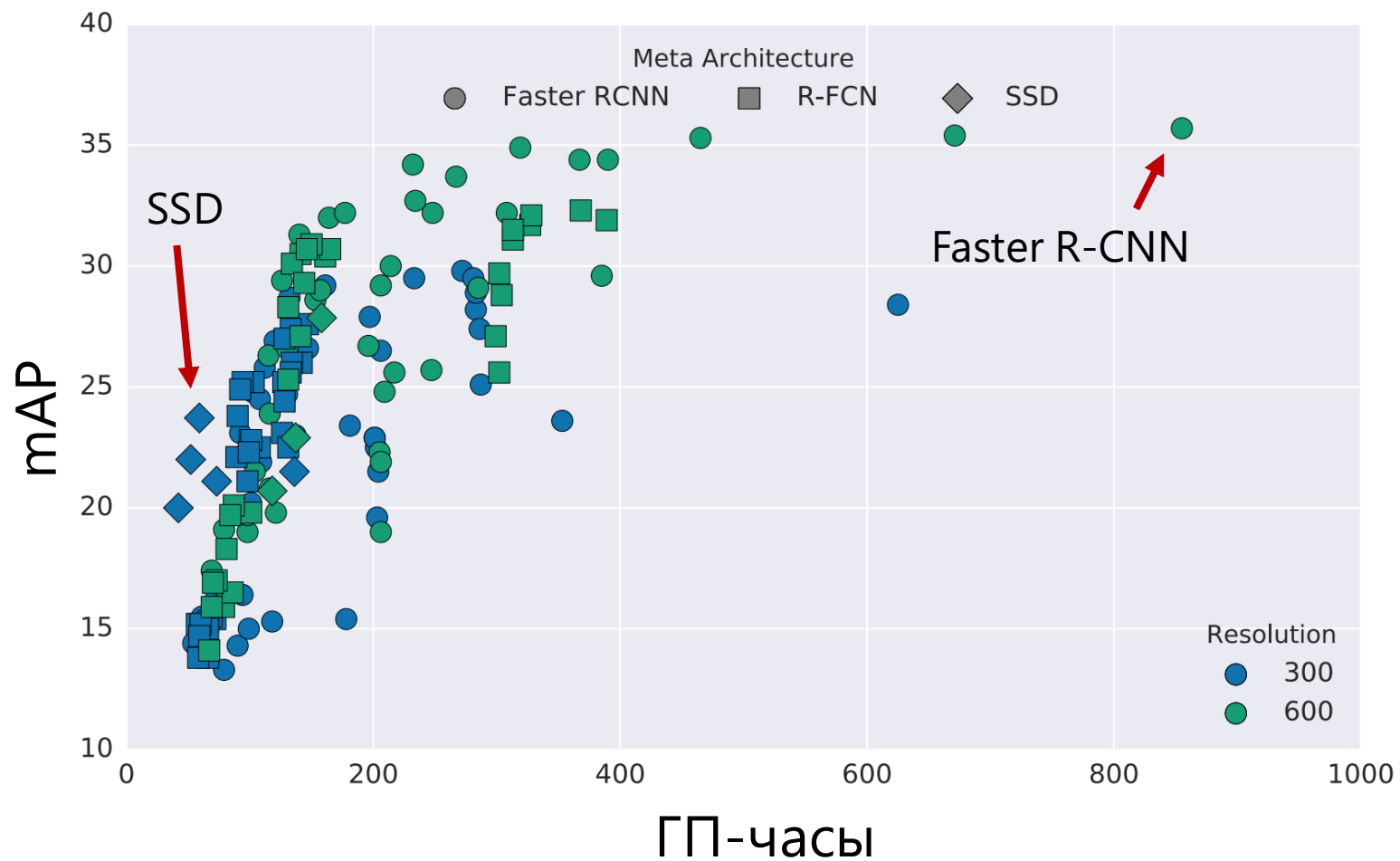
Пример



Сравнение алгоритмов



Сравнение алгоритмов



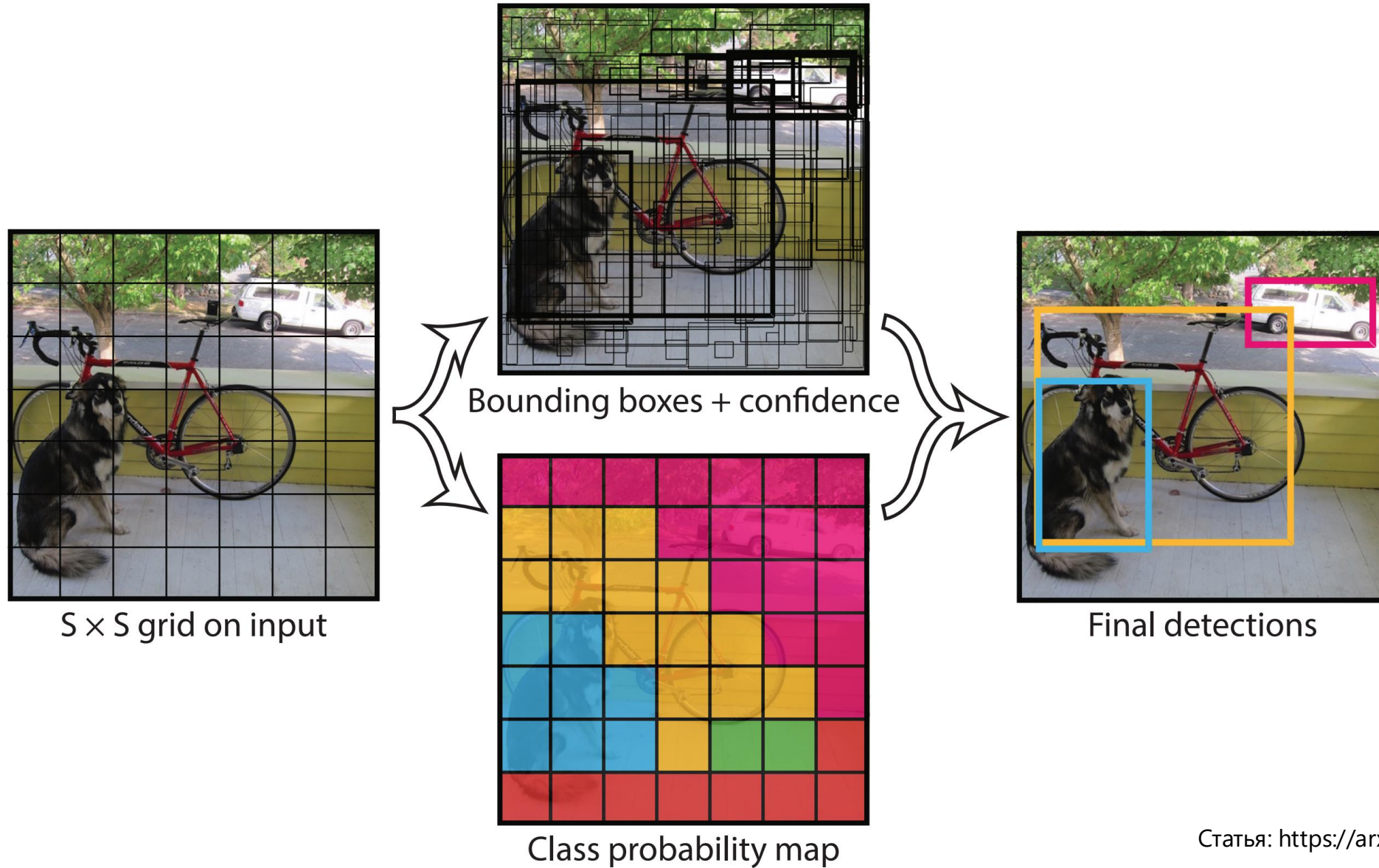
Сравнение алгоритмов

- ▶ Faster R-CNN лучше по качеству, но медленнее
- ▶ SSD быстрее, но хуже по качеству



YOLO: You Only Look Once

YOLO: идея

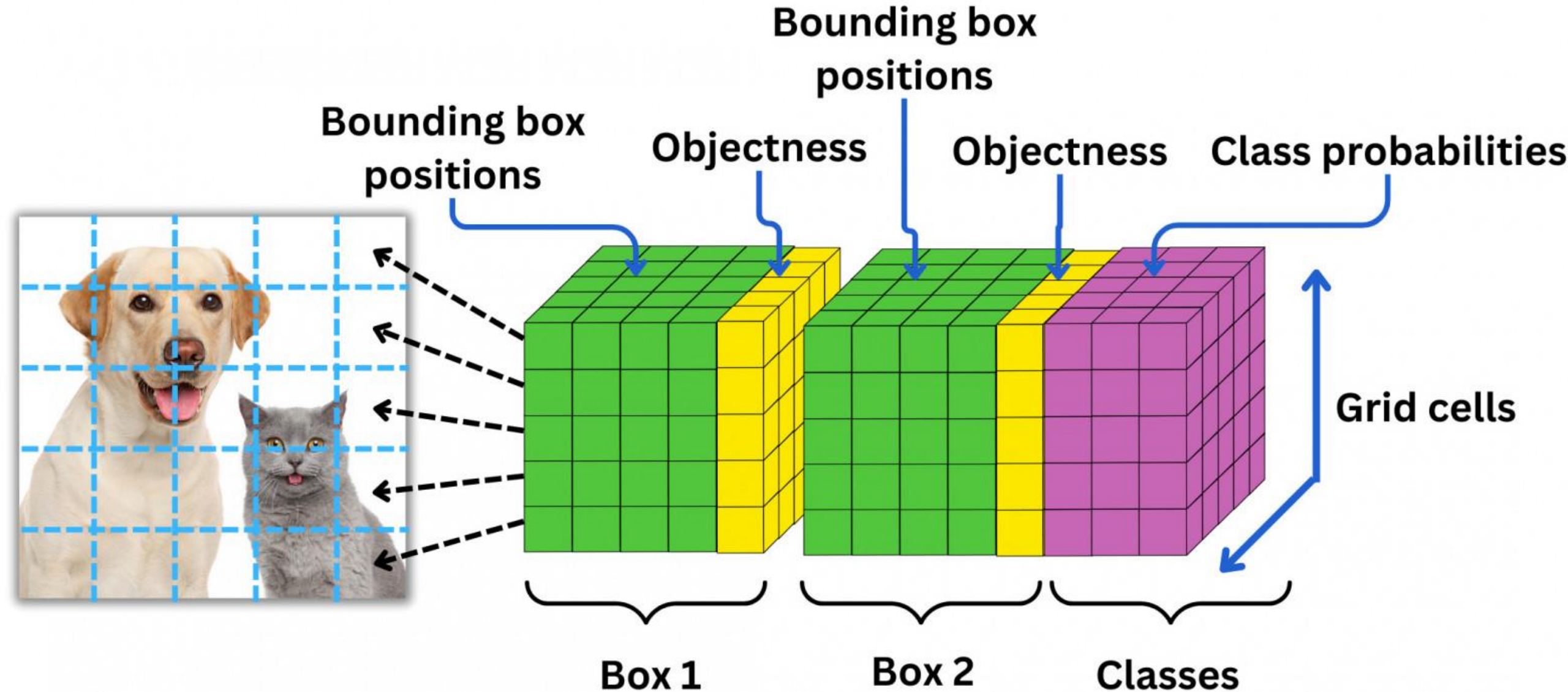


Статья: <https://arxiv.org/pdf/1506.02640>

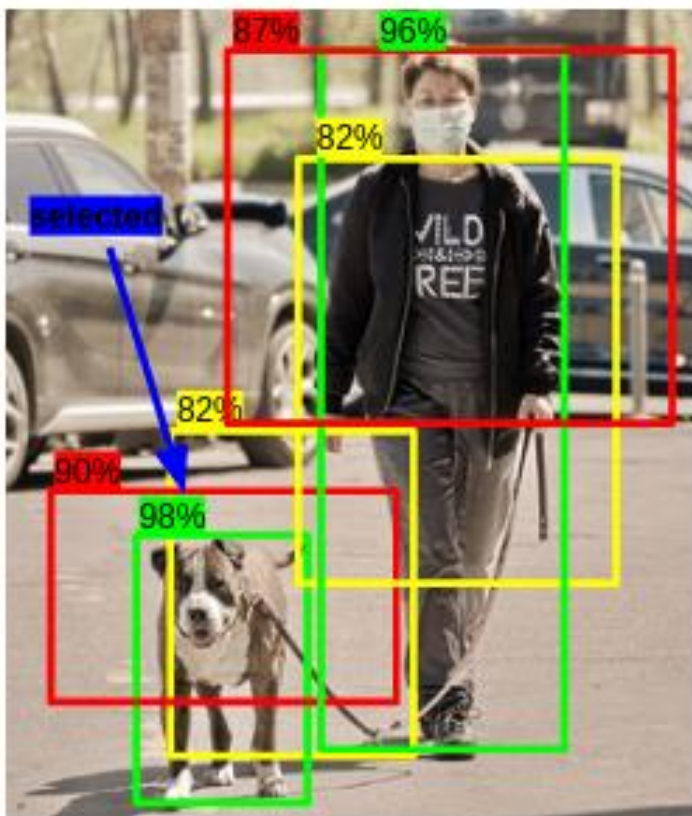
YOLO: идея

- ▶ Изображение делим решеткой на $S \times S$ ($S = 7$) ячеек. Если центр объекта падает в ячейку, то эта ячейка отвечает за детектирование этого объекта.
- ▶ Для каждой ячейки предсказываем B ($B = 2$) прямоугольников (bounding boxes).
 - Для каждого прямоугольника предсказывается вероятности, что объекты находится в этом прямоугольнике.
 - Для каждого прямоугольника предсказываем также x, y, w, h (координаты центра, высоту и ширину области с объектом).
 - Для прямоугольников считаем уверенность, что прямоугольник содержит объект. Хотим, чтобы эта уверенность была как IoU (pred vs truth). Оставляем только те, у которых IoU выше порога.
- ▶ Применяем к прогнозам Non-maximum suppression.

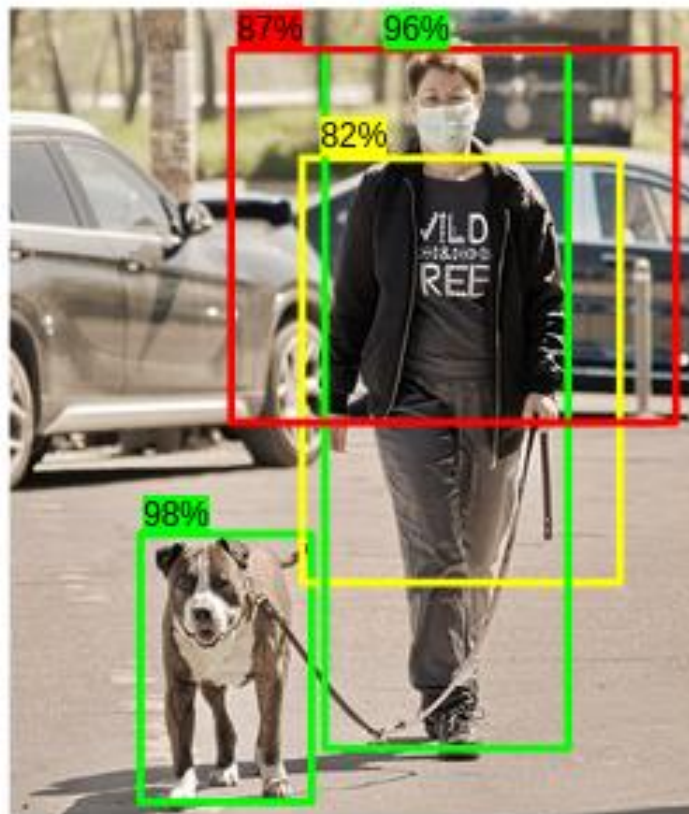
YOLO: прогнозы



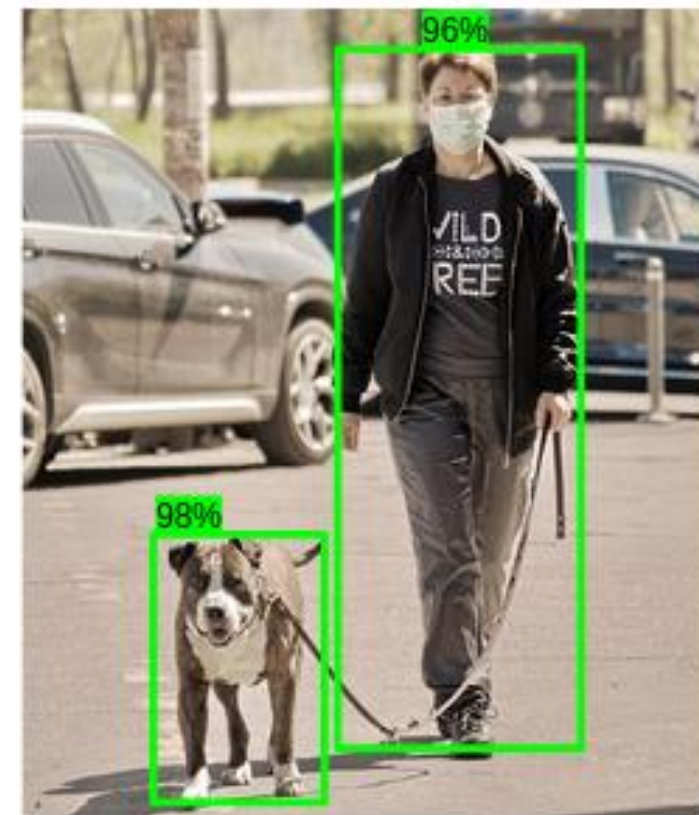
Non-maximum suppression (напоминка)



Step 1: Selecting Bounding box with highest score

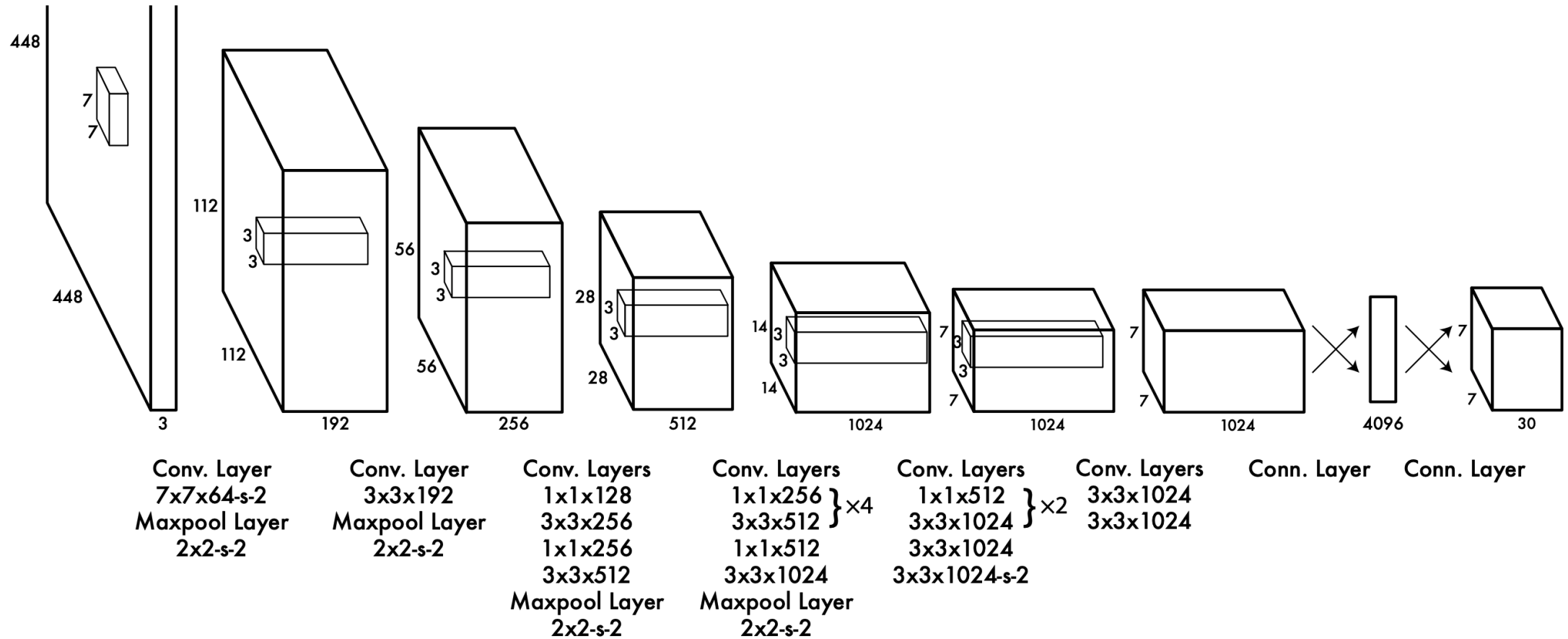


Step 3: Delete Bounding box with high overlap



Step 5: Final Output

YOLO: архитектура сети



Функция потерь

Regression
loss

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

Confidence
loss

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

Classification
loss

$$+ \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

YOLO

YOLO: Release Dates Timeline



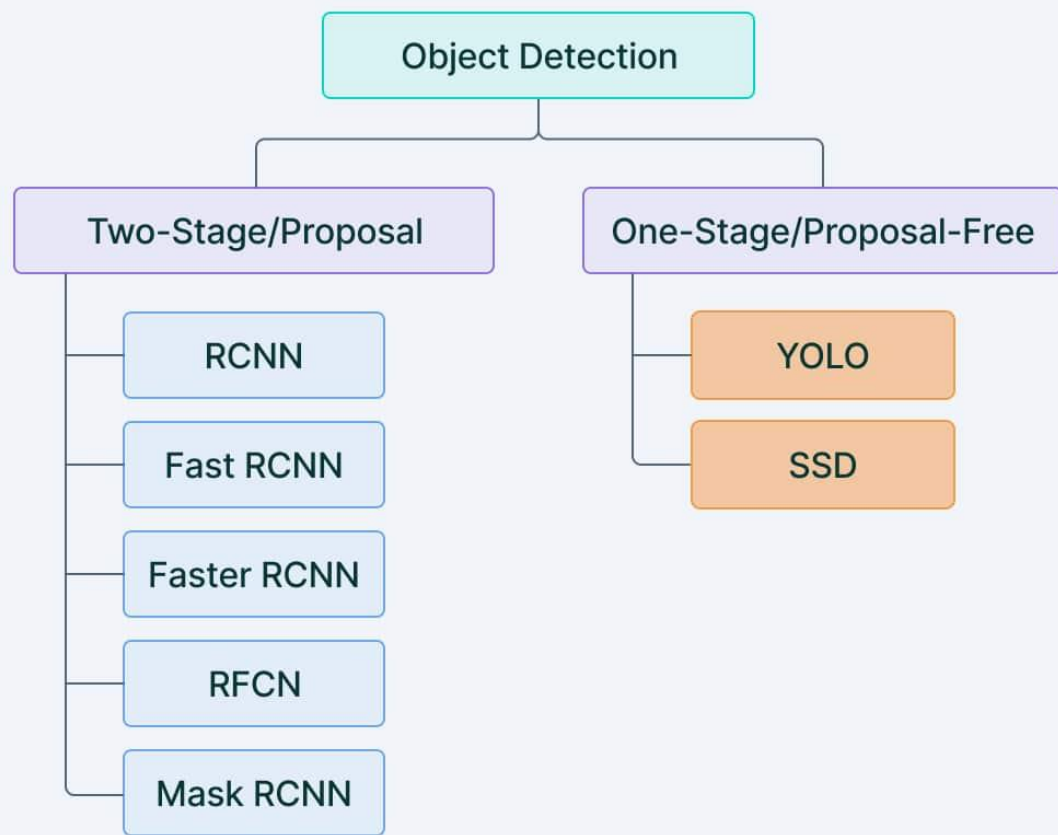
YOLOv11
2024

ENCORD

Другие алгоритмы

- ▶ Двухэтапные:
 - R-FCN
 - Cascade R-CNN
 - Hybrid Task Cascade
- ▶ Одноэтапные
 - YOLO - YOLOv12
 - RetinaNet
 - FCOS
 - CentripetalNet

One and two stage detectors



Поставьте свою оценку лекции

